

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота з освітнього компоненту
«Моделювання систем. Курсова робота»
Тема: «Імітаційна модель комплектувального конвеєра автоматизованого складального цеху на основі формального опису стохастичною мережею Петрі»

Керівник:
Коринецький Д. С.

«Допущено до захисту»
_____

«___» _____ 2026 р.

Захищено з оцінкою
_____

Члени комісії:

_____ Стельмах О. П.

_____ Коринецький Д. С.

Виконавець:
Ломакін Максим Сергійович
студент групи ІП-33
залікова книжка № ___
_____

«16» листопада 2026 р.

Київ – 2026

ЗМІСТ

АНОТАЦІЯ 4
ВСТУП 6
ЗАВДАННЯ 8
1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ 9
1.1 Мета моделювання 9
1.2 Складові елементи системи 9
1.3 Схематичне зображення процесу комплектації 10
1.4 Вхідні змінні моделі 11
1.5 Параметри моделі 12
1.6 Вихідні характеристики моделі 13
1.7 Припущення та спрощення моделі 13
1.8 Висновок до розділу 14
2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ 15
2.1 Елементи стохастичної мережі Петрі 15
2.2 Позиції мережі та початкове маркування 15
2.3 Переходи мережі та розв’язання конфлікту 16
2.4 Графічне зображення мережі Петрі 17
2.5 Мережа Петрі з обмеженим накопичувачем 17
2.6 Генерування випадкових величин 19
2.7 Формули обчислення вихідних характеристик 19
2.8 Висновок до розділу 21
3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ 22
3.1 Вибір засобів розробки 22
3.2 Алгоритм імітації стохастичної мережі Петрі 22
3.3 Структура програми 25
3.4 Просування модельного часу та зміна стану мережі 27
3.5 Обчислення вихідних характеристик 30
3.6 Верифікація моделі 32

3.7 Висновок до розділу 36
4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ 38
4.1 Мета експерименту, фактор і відгуки 38
4.2 Теоретичний розрахунок характеристик системи 38
4.3 Перехідний період і вибір часу моделювання 41
4.4 Визначення кількості прогонів 44
4.5 Статистичні оцінки характеристик базової моделі 45
4.6 Дисперсійний аналіз варіантів комплектації 46
4.7 Перехідний період і кількість прогонів моделі з накопичувачем 48
4.8 Вплив місткості накопичувача на ймовірність пропуску секції 50
4.9 Висновок до розділу 52
5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ 53
5.1 Характеристики роботи конвеєра 53
5.2 Доцільність переходу на комплектацію по 20 виробів 53
5.3 Обмежений накопичувач з блокуванням постачання 54
5.4 Напрями подальших досліджень 56
ВИСНОВКИ 57
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 59
ДОДАТОК А. Код алгоритму імітації 60
ДОДАТОК Б. Код верифікації моделі 71
ДОДАТОК В. Код визначення перехідного періоду та кількості прогонів 79
ДОДАТОК Г. Код експериментального дослідження 84
ДОДАТОК Д. Фрагмент протоколу подій 93
ДОДАТОК Е. Результати перебору місткостей накопичувача 94

АНОТАЦІЯ

Ломакін Максим Сергійович. Курсова робота на тему «Імітаційна модель комплектувального конвеєра автоматизованого складального цеху на основі формального опису стохастичною мережею Петрі».

Текстова частина курсової роботи складається із вступу, 5 розділів, висновків, списку використаних джерел з 8 найменувань та 6 додатків, які викладено на 94 сторінках. В тексті роботи оформлено 11 рисунків, 20 таблиць.

У роботі поставлено завдання оцінити ймовірність пропуску секції комплектувального конвеєра і характеристики черг виробів двох типів та дослідити доцільність комплектації секції по 20 виробів.

Імітаційна модель розроблена на основі опису системи стохастичною мережею Петрі; власний алгоритм імітації з просуванням часу за найближчою подією реалізовано мовою Python, його коректність перевірено на 7 наборах вхідних даних, для яких очікувані значення обчислено аналітично.

Проведено дослідження перехідного періоду і кількості прогонів, дисперсійний аналіз варіантів комплектації та регресійний аналіз впливу місткості накопичувача.

Результати моделювання свідчать, що за $T_{\text{мод}}$ = 480 хв (1600 прогонів) ймовірність пропуску секції становить 0,0589 у базовому варіанті та 0,0537 у варіанті по 20 виробів; різниця статистично значуща, але практично мала, тоді як середні черги у другому варіанті більші на 3,3 і 3,2 виробу. Черги базової системи не мають сталого режиму за будь-якої швидкості конвеєра. Запропоновано обмежений накопичувач із блокуванням постачання місткістю 20 і 60 виробів: у сталому режимі ($T_{\text{мод}}$ = 40 000 хв, 120 прогонів) черги не перевищують місткостей накопичувачів, а продуктивність конвеєра зменшується на 1,4 %.

Зроблено висновки щодо недоцільності переходу на комплектацію по 20 виробів і щодо умов застосування накопичувача.

Ключові слова: імітаційна модель, алгоритм імітації, стохастична мережа Петрі, комплектувальний конвеєр, дисперсійний аналіз, регресійний аналіз, Python.

ВСТУП

Комплектувальні конвеєри застосовують у складальних цехах для подачі до робочих місць наборів деталей, необхідних для однієї операції збирання. Деталі різних типів надходять від окремих постачальників партіями та в різні моменти часу, а секція конвеєра може бути укомплектована лише за наявності потрібної кількості деталей кожного типу. Невідповідність між моментами надходження і моментами подачі секцій призводить до двох видів втрат: секції, що проходять порожніми, зменшують продуктивність лінії, а деталі, які очікують на комплектацію, утворюють незавершене виробництво і займають складські площі.

Аналітичні методи теорії масового обслуговування погано пристосовані до такої системи: надходження відбуваються партіями з рівномірно розподіленими інтервалами, конвеєр рухається детерміновано, а умова комплектації вимагає одночасної наявності ресурсів двох типів. Тому для оцінювання характеристик системи та порівняння варіантів її організації використано імітаційне моделювання [1, 2].

Метою роботи є побудова імітаційної моделі комплектувального конвеєра, визначення ймовірності пропуску секції та характеристик черг виробів, оцінювання доцільності переходу на комплектацію секції по 20 виробів кожного типу і розроблення пропозицій щодо поліпшення роботи системи.

Для формального опису системи обрано стохастичну мережу Петрі, оскільки цей формалізм природно відображає умову одночасної наявності кількох ресурсів, пакетні надходження (кратні дуги) та конфлікт між альтернативними подіями (пріоритети переходів) [3]. Алгоритм імітації мережі Петрі розроблено самостійно з просуванням модельного часу за найближчою подією. Для оцінювання результатів застосовано методи математичної статистики: довірчі інтервали, однофакторний дисперсійний аналіз і регресійний аналіз.

Програмну реалізацію виконано мовою Python 3.12 [4] із використанням бібліотек NumPy для генерування випадкових чисел [5], SciPy для статистичних розрахунків [6] і Matplotlib для побудови графіків [7].

ЗАВДАННЯ

Виконати дослідження для такої системи:

На комплектувальний конвеєр автоматизованого складального цеху надходять 5 виробів першого типу кожні 5±1 хвилин та 20 виробів другого типу кожні 20±7 хвилин. Конвеєр рухається зі швидкістю 1 секція / 10 хвилин. За наявності 10 деталей кожного типу секція заповнюється деталями і комплектація вважається завершеною. При нестачі деталей секція конвеєра залишається порожньою.

Визначити ймовірність пропуску секції, середні та максимальні значення черги по кожному типу виробів. Дослідити доцільність переходу на комплектацію в одній секції конвеєра по 20 виробів кожного типу. Швидкість руху конвеєра при цьому змінюється відповідно до 1 секції / 20 хвилин.

Для вищезазначеної системи:

1) розробити опис концептуальної моделі системи;

2) виконати формалізацію опису об’єкта моделювання в термінах визначеного у завданні формалізму;

3) розробити алгоритм імітації моделі дискретно-подійної системи у відповідності до побудованого формального опису;

4) для доведення коректності побудованого алгоритму виконати верифікацію алгоритму імітації;

5) визначити статистичні оцінки заданих характеристик моделі, що є метою моделювання;

6) провести експериментальне дослідження моделі;

7) інтерпретувати результати моделювання та сформулювати пропозиції щодо поліпшення функціонування системи;

8) зробити висновки щодо складності розробки моделі та алгоритму імітації на основі використаного формалізму, отриманих результатів моделювання та їх корисності.

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

1.1 Мета моделювання

Об’єктом моделювання є пост комплектації складального цеху, до якого надходять вироби двох типів від двох постачальників і повз який рухається комплектувальний конвеєр. Кожна секція конвеєра, що підходить до поста, або отримує комплект із однакової кількості виробів кожного типу, або проходить порожньою.

Мета моделювання полягає в тому, щоб:

– оцінити ймовірність пропуску секції конвеєра;

– оцінити середні та максимальні довжини черг виробів кожного типу, що очікують на комплектацію;

– порівняти базовий варіант комплектації (10 виробів кожного типу в секції, секція кожні 10 хв) з варіантом по 20 виробів кожного типу і секцією кожні 20 хв та зробити висновок про доцільність переходу;

– запропонувати зміну організації поста, що поліпшує його роботу, і кількісно оцінити її ефект.

1.2 Складові елементи системи

Система складається з таких елементів:

– постачальник виробів типу 1 – передає на пост партії по 5 виробів; інтервал між партіями випадковий, від 4 до 6 хв;

– постачальник виробів типу 2 – передає партії по 20 виробів з інтервалом від 13 до 27 хв;

– накопичувачі виробів типу 1 і типу 2 – місця біля поста, де вироби очікують на комплектацію; вироби одного типу взаємозамінні, тому накопичувач характеризується лише кількістю виробів;

– комплектувальний конвеєр – з постійною швидкістю подає до поста порожні секції, у базовому варіанті одну секцію кожні 10 хв;

– пост комплектації – у момент подачі секції перевіряє наявність виробів і або укладає в секцію по 10 виробів кожного типу, або пропускає секцію порожньою.

Постачальники працюють незалежно один від одного і від конвеєра. Взаємодія елементів відбувається лише через накопичувачі: постачальники їх поповнюють, пост комплектації одночасно забирає з обох накопичувачів однакову кількість виробів.

1.3 Схематичне зображення процесу комплектації

Процес функціонування системи зображено на рисунку 1.1. Він складається з трьох паралельних циклічних процесів. Кожен постачальник виготовляє партію виробів, передає її в накопичувач і відразу починає виготовлення наступної партії. Конвеєр через рівні проміжки часу подає до поста порожню секцію; у момент подачі перевіряється умова комплектації: якщо в кожному накопичувачі є щонайменше 10 виробів, по 10 виробів кожного типу переходять у секцію, інакше секція залишає пост порожньою і вироби з накопичувачів не забираються.

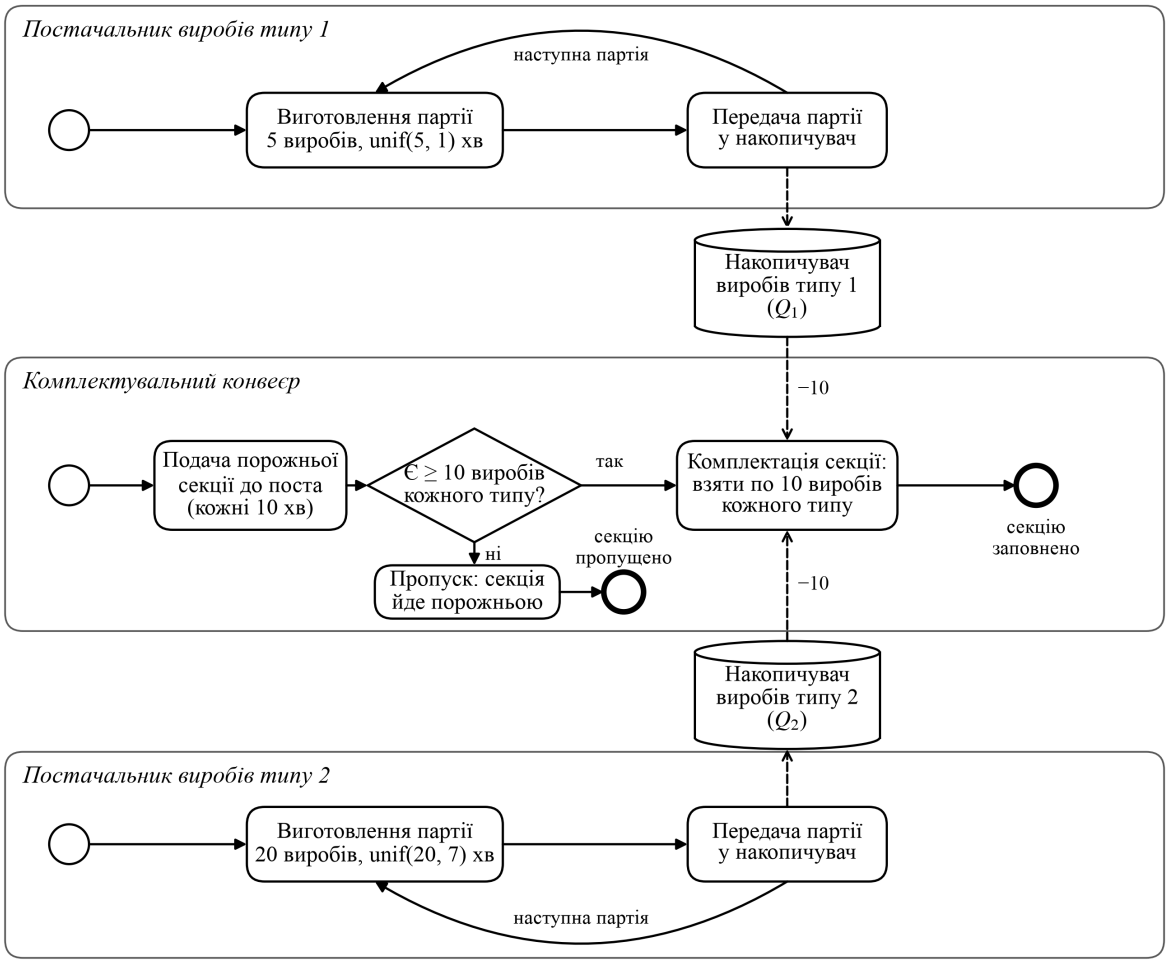


Рисунок 1.1 – Схема процесу комплектації секцій конвеєра

1.4 Вхідні змінні моделі

Вхідними змінними моделі є випадкові інтервали між надходженнями партій і початковий стан накопичувачів (табл. 1.1). Інтервали задано в позначеннях [3]: *unif*(*d*, *s*) – випадкова величина, рівномірно розподілена на інтервалі (*d* − *s*; *d* + *s*).

Таблиця 1.1 – Вхідні змінні моделі

Змінна	Позначення	Значення в базовій моделі	Закон розподілу
Інтервал між партіями виробів типу 1, хв	τ_1	unif(5, 1), від 4 до 6	рівномірний
Інтервал між партіями виробів типу 2, хв	τ_2	unif(20, 7), від 13 до 27	рівномірний
Початкова кількість виробів типу 1 у накопичувачі, од.	$Q_1(0)$	0	детермінована
Початкова кількість виробів типу 2 у накопичувачі, од.	$Q_2(0)$	0	детермінована

Порожні накопичувачі в початковий момент відповідають початку робочої зміни. Під час верифікації моделі інтервали надходжень змінюються, зокрема замінюються детермінованими значеннями, що дає змогу порівняти результати з ручним розрахунком.

1.5 Параметри моделі

Параметри моделі для базового варіанта, варіанта комплектації по 20 виробів і модифікації з обмеженим накопичувачем, яку запропоновано в розділі 5, наведено в таблиці 1.2.

Таблиця 1.2 – Параметри моделі

Параметр	Позна-чення	Базовий варіант	Варіант 20/20	Модель з накопичувачем
Кількість виробів у партії типу 1, од.	b_1	5	5	5
Кількість виробів у партії типу 2, од.	b_2	20	20	20
Кількість виробів кожного типу в секції, од.	k	10	20	10
Період подачі секцій, хв	T_c	10	20	10
Місткість накопичувача типу 1, од.	L_1	не обмежена	не обмежена	15…30
Місткість накопичувача типу 2, од.	L_2	не обмежена	не обмежена	30…80
Час моделювання, хв	$T_{мод}$	480; 2400	480; 2400	40 000
Тривалість перехідного періоду, хв	$T_п$	0	0	3000

Середня інтенсивність надходження виробів типу 1 дорівнює 5 / 5 = 1 виріб/хв, типу 2 – 20 / 20 = 1 виріб/хв. Конвеєр за одиницю часу може забрати не більше 10 / 10 = 1 виробу кожного типу за хвилину, і в варіанті 20/20 ця величина та сама. Отже, середнє надходження дорівнює максимальній швидкості комплектації; наслідки цього для черг розглянуто в підрозділі 4.2.

Часи моделювання 480 хв (одна робоча зміна) і 2400 хв (п’ять змін) застосовано для базового варіанта та варіанта 20/20, вибір цих значень обґрунтовано в розділі 4. Модель з накопичувачем має сталий режим, тому для неї визначено перехідний період і довжину прогону.

1.6 Вихідні характеристики моделі

Відповідно до завдання основними вихідними характеристиками є:

– ймовірність пропуску секції P_{skip} – частка секцій, що пройшли пост порожніми, серед усіх секцій, поданих за час моделювання;

– середня довжина черги виробів типу 1 $\bar{Q}_1$ і типу 2 $\bar{Q}_2$ – середня за часом кількість виробів у накопичувачі;

– максимальна довжина черги виробів типу 1 $Q_{1,max}$ і типу 2 $Q_{2,max}$.

Для інтерпретації результатів і порівняння варіантів додатково визначаються:

– продуктивність конвеєра $П$ – кількість виробів кожного типу, укомплектованих за годину;

– середній час очікування виробу типу i на комплектацію $\bar{W}_i$;

– у моделі з накопичувачем – частка часу B_i, протягом якого постачальник типу i простоює через відсутність вільного місця.

Для обчислення цих характеристик під час імітації реєструються кількості заповнених і пропущених секцій, кількості партій, що надійшли, і зміна довжин черг у часі. Формули обчислення наведено в підрозділі 2.7.

1.7 Припущення та спрощення моделі

Під час побудови моделі прийнято такі припущення:

1) одиниця модельного часу – 1 хв; партія надходить у накопичувач цілком в один момент часу;

2) інтервали між партіями незалежні між собою і не залежать від стану системи; перша партія кожного типу надходить через випадковий інтервал від початку моделювання, а перша секція подається в момент $t = T_c$;

3) комплектація миттєва: рішення про заповнення або пропуск секції приймається в момент її подачі, і секція одразу залишає пост;

4) неповних комплектів не буває: якщо хоча б одного типу виробів менше за k, секція пропускається, а вироби залишаються в накопичувачах;

5) якщо в один момент часу відбувається кілька подій, спочатку враховуються всі надходження партій і подача секції, а потім приймається рішення про комплектацію;

6) у базовій моделі місткість накопичувачів не обмежена; у моделі з накопичувачем постачальник починає виготовлення партії лише тоді, коли для неї є вільне місце, інакше чекає.

Час виготовлення партії і час її транспортування до поста не розділяються: модель відтворює лише моменти надходження партій. Відмови обладнання та перерви в роботі не розглядаються.

1.8 Висновок до розділу

Визначено мету моделювання, склад елементів системи та їхню взаємодію, побудовано схему процесу комплектації. Сформульовано вхідні змінні, параметри трьох варіантів моделі та вихідні характеристики, що відповідають завданню. Показано, що середня інтенсивність надходження виробів кожного типу збігається з максимальною швидкістю комплектації, що визначає особливості подальшого дослідження. Прийняті припущення однозначно задають поведінку системи в моменти одночасних подій.

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ

2.1 Елементи стохастичної мережі Петрі

Стохастична мережа Петрі складається з позицій, переходів і дуг, що з'єднують позиції з переходами [1, 3]. Стан мережі задається маркуванням – кількістю маркерів у кожній позиції. Кожна дуга має кратність – кількість маркерів, що переміщуються по ній за одне спрацювання переходу.

Перехід може спрацювати, якщо в кожній його вхідній позиції кількість маркерів не менша за кратність відповідної дуги. Спрацювання складається з двох подій: вхід маркерів (маркери вилучаються з вхідних позицій у поточний момент часу) і вихід маркерів (маркери додаються у вихідні позиції після часової затримки переходу). Затримка може бути детермінованою або випадковою величиною із заданим законом розподілу. Перехід може спрацювати повторно ще до виходу маркерів попереднього спрацювання, тобто він є багатоканальним.

Якщо кілька переходів претендують на одні й ті самі маркери, виникає конфлікт. Його розв'язують за пріоритетом *r*: спрацьовує перехід з найбільшим пріоритетом, а серед переходів з однаковим пріоритетом – випадково, з імовірностями *b*.

На рисунках мережі позицію позначено колом з кількістю маркерів, перехід – прямокутником. Над переходом наведено його назву, під ним – затримку і пару «*r*, *b*». Кратність дуги, більша за 1, вказана числом біля перекресленої дуги.

2.2 Позиції мережі та початкове маркування

Позиції мережі базової моделі та їхнє початкове маркування наведено в таблиці 2.1.

Таблиця 2.1 – Позиції мережі Петрі

Позиція	Зміст	Початкове маркування
G_1	постачальник виробів типу 1 готовий почати виготовлення партії	1
G_2	постачальник виробів типу 2 готовий почати виготовлення партії	1

Продовження таблиці 2.1

Позиція	Зміст	Початкове маркування
Q_1	вироби типу 1 у накопичувачі	0
Q_2	вироби типу 2 у накопичувачі	0
C	привід конвеєра готовий подати наступну секцію	1
S	порожня секція біля поста комплектації	0
N_{fill}	лічильник заповнених секцій	0
N_{skip}	лічильник пропущених секцій	0

Позиції G_1, G_2 і C містять по одному маркеру і з'єднані з відповідним переходом дугами в обох напрямках. Маркер повертається в позицію лише після виходу маркерів з переходу, тому переходи «Надходження 1», «Надходження 2» і «Рух конвеєра» мають не більше одного активного каналу: наступна партія або наступна секція з'являється лише після завершення попередньої. Позиції N_{fill} і N_{skip} не впливають на роботу мережі і слугують для обчислення вихідних характеристик.

2.3 Переходи мережі та розв'язання конфлікту

Параметри переходів базової моделі наведено в таблиці 2.2. Затримки вказано у хвилинах.

Таблиця 2.2 – Переходи мережі Петрі базової моделі

Перехід	Вхідні позиції (кратність)	Вихідні позиції (кратність)	Затримка, хв	r	b
Надходження 1	G_1 (1)	G_1 (1), Q_1 (5)	unif(5, 1)	0	1,0
Надходження 2	G_2 (1)	G_2 (1), Q_2 (20)	unif(20, 7)	0	1,0
Рух конвеєра	C (1)	C (1), S (1)	10	0	1,0
Комплектація	S (1), Q_1 (10), Q_2 (10)	N_{fill} (1)	0	1	1,0
Пропуск секції	S (1)	N_{skip} (1)	0	0	1,0

Переходи «Комплектація» і «Пропуск секції» мають спільну вхідну позицію S, тобто перебувають у конфлікті за маркер секції. Обидва мають нульову затримку, тому рішення щодо секції приймається в момент її подачі. «Комплектація» має пріоритет 1, «Пропуск секції» – пріоритет 0. Якщо в Q_1 і Q_2 є щонайменше по 10 маркерів, активні обидва переходи і спрацьовує «Комплектація»; інакше активний лише «Пропуск секції». Так засобами мережі

Петрі відтворено правило завдання: секція залишається порожньою тоді й лише тоді, коли деталей хоча б одного типу недостатньо.

Маркер секції з'являється в позиції S в той самий момент, коли можуть завершитися надходження партій. Щоб рішення про секцію враховувало всі вироби, що надійшли в цей момент, алгоритм імітації спочатку виконує всі виходи маркерів, а потім – входи (підрозділ 3.2).

2.4 Графічне зображення мережі Петрі

Мережу Петрі базової моделі зображено на рисунку 2.1.

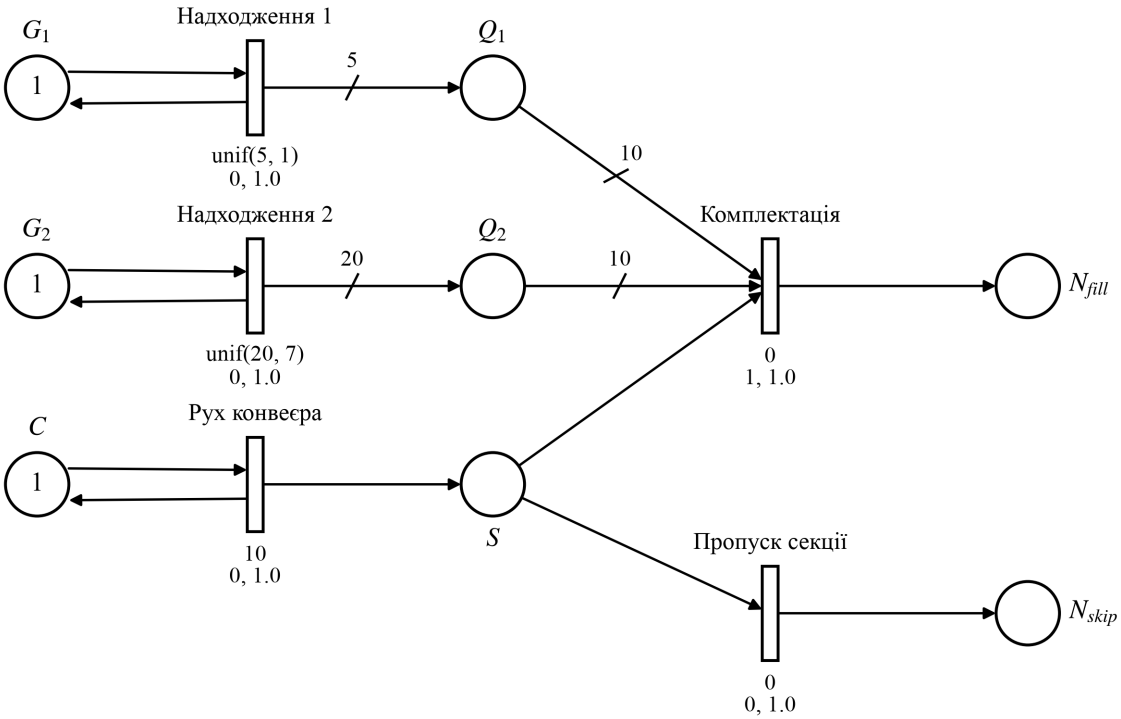


Рисунок 2.1 – Стохастична мережа Петрі базової моделі

Варіант комплектації по 20 виробів має ту саму структуру мережі. Змінюються лише кратності дуг з позицій Q_1 і Q_2 до переходу «Комплектація» (20 замість 10) і затримка переходу «Рух конвеєра» (20 хв замість 10 хв). Тому обидва варіанти описуються однією мережею з параметрами k і T_c.

2.5 Мережа Петрі з обмеженим накопичувачем

У базовій моделі накопичувачі не обмежені, і, як показано в підрозділі 4.2, черги виробів у ній не мають сталого режиму. Для дослідження пропозиції щодо

поліпшення системи мережу доповнено позиціями F_1 і F_2, які містять вільні місця накопичувачів виробів типу 1 і типу 2. Початкове маркування цих позицій дорівнює місткостям L_1 і L_2. Зміни переходів наведено в таблиці 2.3, решта переходів і позицій не змінюється.

Таблиця 2.3 – Переходи, змінені в мережі з накопичувачем

Перехід	Вхідні позиції (кратність)	Вихідні позиції (кратність)
Надходження 1	G_1 (1), F_1 (5)	G_1 (1), Q_1 (5)
Надходження 2	G_2 (1), F_2 (20)	G_2 (1), Q_2 (20)
Комплектація	S (1), Q_1 (10), Q_2 (10)	N_{fill} (1), F_1 (10), F_2 (10)

Мережу з накопичувачем для $L_1 = 20$ і $L_2 = 60$ зображено на рисунку 2.2.

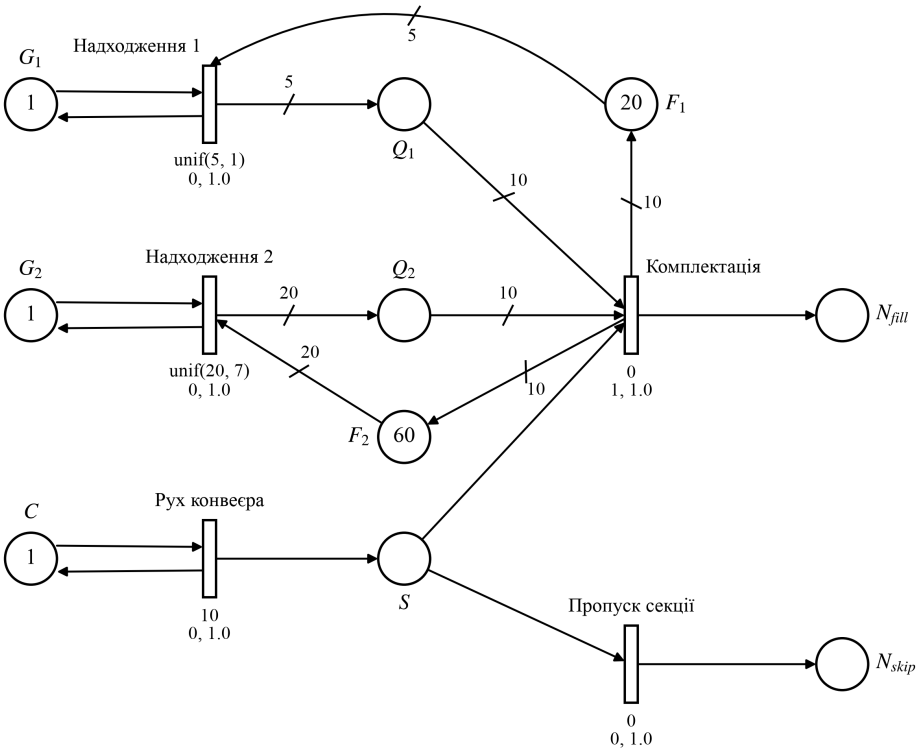


Рисунок 2.2 – Стохастична мережа Петрі з обмеженим накопичувачем

Постачальник починає виготовлення партії лише тоді, коли в накопичувачі є вільні місця для всієї партії, і резервує їх на початку виготовлення. Якщо місць недостатньо, маркер залишається в позиції G_i, і постачальник простоює до найближчої комплектації, яка звільняє по 10 місць кожного типу. Для кожного типу виконується інваріант $M(Q_i) + M(F_i) + b_i \cdot n_i = L_i$, де n_i – кількість партій типу i, що виготовляються (0 або 1). Отже, черга виробів типу i ніколи не перевищує

L_i. Місткість має бути не меншою за розмір партії і кількість виробів у секції, $L_i \geq \max(b_i, k)$, інакше мережа потрапляє в тупиковий стан.

2.6 Генерування випадкових величин

Випадкові затримки переходів «Надходження 1» і «Надходження 2» мають рівномірний розподіл. Значення рівномірно розподіленої величини обчислюється за формулою

$$\tau = d - s + 2s \cdot r \tag{2.1}$$

де τ – значення затримки, хв;

d – середнє значення затримки, хв;

s – половина ширини інтервалу можливих значень, хв;

r – випадкове число, рівномірно розподілене на інтервалі (0; 1).

Випадкові числа r отримуються з генератора PCG64 бібліотеки NumPy [5]. Кожен прогін моделі використовує власне початкове значення генератора, що робить результати відтворюваними, а прогони – незалежними. Затримки переходів «Рух конвеєра», «Комплектація» і «Пропуск секції» детерміновані.

2.7 Формули обчислення вихідних характеристик

Вихідні характеристики обчислюються на інтервалі спостереження від $T_{\text{п}}$ до $T_{\text{мод}}$. Для базової моделі $T_{\text{п}} = 0$. Ймовірність пропуску секції визначається як частка пропущених секцій:

$$P_{skip} = \frac{N_{skip}}{N_{fill} + N_{skip}} \tag{2.2}$$

де N_{fill} – кількість спрацювань переходу «Комплектація» на інтервалі спостереження;

N_{skip} – кількість спрацювань переходу «Пропуск секції» на інтервалі спостереження.

Довжина черги між двома послідовними подіями не змінюється, тому середня довжина черги обчислюється як середнє, зважене за тривалістю інтервалів незмінного маркування:

$$\overline{Q_i} = \frac{1}{T_{\text{мод}} - T_{\text{п}}} \sum_{j=1}^{n} M_j(Q_i) \cdot \Delta t_j \qquad (2.3)$$

де $M_j(Q_i)$ – кількість маркерів у позиції Q_i на j-му інтервалі незмінного маркування, од.;

Δt_j – тривалість j-го інтервалу, хв;

n – кількість інтервалів незмінного маркування на інтервалі спостереження;

i – тип виробів, i = 1, 2.

Середнє квадратичне відхилення довжини черги, зважене за часом:

$$\sigma_i = \sqrt{\frac{1}{T_{\text{мод}} - T_{\text{п}}} \sum_{j=1}^{n} M_j(Q_i)^2 \cdot \Delta t_j - \overline{Q_i}^2} \qquad (2.4)$$

Максимальна довжина черги визначається серед значень, які зберігалися протягом ненульового часу:

$$Q_{i,\max} = \max_j M_j(Q_i) \qquad (2.5)$$

Значення, що існують лише всередині одного моменту часу (наприклад, черга після надходження партії, але до комплектації секції, поданої в той самий момент), не враховуються, оскільки фізично вироби в такому стані не перебувають.

Продуктивність конвеєра в виробах кожного типу за годину:

$$П = \frac{60 \cdot k \cdot N_{fill}}{T_{\text{мод}} - T_{\text{п}}} \qquad (2.6)$$

де k – кількість виробів кожного типу в секції, од.;

60 – кількість хвилин в годині.

Середній час очікування виробу на комплектацію обчислюється за формулою Літтла через середню довжину черги і фактичну інтенсивність надходження:

$$\overline{W_i} = \frac{\overline{Q_i}}{\lambda_i}, \quad \lambda_i = \frac{b_i \cdot A_i}{T_{\text{мод}} - T_{\text{п}}} \tag{2.7}$$

де λ_i – інтенсивність надходження виробів типу i, виробів/хв;

b_i – кількість виробів у партії типу i, од.;

A_i – кількість партій типу i, що надійшли на інтервалі спостереження.

У моделі з накопичувачем маркер у позиції G_i затримується лише тоді, коли постачальник очікує вільних місць. Частка часу простою постачальника дорівнює середньому за часом маркуванню цієї позиції:

$$B_i = \frac{1}{T_{\text{мод}} - T_{\text{п}}} \sum_{j=1}^{n} M_j(G_i) \cdot \Delta t_j \tag{2.8}$$

де $M_j(G_i)$ – кількість маркерів у позиції G_i на j-му інтервалі, 0 або 1.

Формули (2.2)–(2.5) визначають характеристики, задані в завданні, формули (2.6)–(2.8) – додаткові характеристики для порівняння варіантів.

2.8 Висновок до розділу

Систему формалізовано стохастичною мережею Петрі з восьми позицій і п’яти переходів. Пакетні надходження відображено кратністю дуг, однаковий склад комплекту – дугами однакової кратності з обох накопичувачів, правило пропуску секції – пріоритетом переходу «Комплектація» над переходом «Пропуск секції». Варіант по 20 виробів відрізняється лише двома параметрами мережі. Для дослідження пропозиції побудовано мережу з обмеженими накопичувачами, у якій вільні місця подано маркерами окремих позицій. Для всіх вихідних характеристик наведено формули їх обчислення за маркуванням мережі.

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ

3.1 Вибір засобів розробки

Програмну реалізацію виконано мовою Python 3.12 [4]. Мова має розвинені бібліотеки для генерування випадкових чисел і статистичної обробки результатів, а інтерпретоване виконання дає змогу швидко змінювати параметри моделі й повторювати експерименти. Використано такі бібліотеки:

– NumPy [5] – генератор випадкових чисел PCG64 з керованим початковим значенням і обчислення над масивами результатів прогонів;

– SciPy [6] – квантилі розподілів Стьюдента і Фішера, перевірка власної реалізації дисперсійного аналізу;

– Matplotlib [7] – побудова графіків;

– pytest – автоматизовані тести алгоритму імітації і моделі.

Алгоритм імітації стохастичної мережі Петрі розроблено самостійно, готові системи моделювання не використовуються. Власна реалізація дає повний контроль над порядком обробки подій, що відбуваються в один момент часу, і над способом збирання статистики, від яких, як показано нижче, залежить результат. Мова Python поступається Java і C++ за швидкодією, але для мережі з восьми–десяти позицій це не є обмеженням: незалежні прогони серії виконуються паралельно в кількох процесах операційної системи (модуль concurrent.futures).

3.2 Алгоритм імітації стохастичної мережі Петрі

Модельний час просувається від події до події. Стан мережі в момент t складається з маркування M і списків E_T моментів виходу маркерів для кожного переходу T; довжина списку дорівнює кількості зайнятих каналів переходу. Найближча подія – найменший момент виходу серед усіх списків. Перед переходом до неї статистика маркування накопичується на інтервалі $[t;\ t_{next})$, протягом якого маркування не змінюється. Зміна стану в момент t складається з двох фаз [3]:

1) фаза виходу – усі переходи, для яких момент виходу дорівнює t, у випадковому порядку додають маркери у свої вихідні позиції;

2) фаза входу – доки існує активний перехід, обирається один перехід з найбільшим пріоритетом (серед рівних за пріоритетом – випадково з імовірностями b), з його вхідних позицій вилучаються маркери, а до списку моментів виходу додається $t + g_T$, де g_T – затримка переходу.

Переходи «Комплектація» і «Пропуск секції» мають нульову затримку, тому їхні виходи припадають на той самий момент t. Після фази входу перевіряється, чи залишилися виходи в поточний момент, і за їх наявності обидві фази повторюються. Просування часу і збирання статистики описує алгоритм 3.1, зміну стану мережі в момент t – алгоритм 3.2.

Алгоритм 3.1 – Просування модельного часу

---

input мережа з множинами позицій P і переходів T, початкове маркування $M(0)$, час моделювання $T_{мод}$, тривалість перехідного періоду $T_п$
$t := 0$; $M := M(0)$; $E_T := \emptyset$ для кожного переходу T
call InPhase(t)
while true **do**
    $t_{next} :=$ min E_T серед усіх переходів
    if $t_{next} > T_{мод}$ **then**
        call Collect(t, $T_{мод}$); $t := T_{мод}$; **break**
    endif
    call Collect(t, t_{next}); $t := t_{next}$
    repeat
        call OutPhase(t)
        call InPhase(t)
    until min $E_T > t$ для всіх переходів
endwhile
output середні, СКВ і максимуми маркування позицій, кількості виходів переходів

procedure Collect(t_1, t_2)
    $\Delta := t_2 - \max(t_1, T_п)$
    if $\Delta > 0$ **then**
        foreach позиція P: $S_P := S_P + M_P \cdot \Delta$; $U_P := U_P + M_P^2 \cdot \Delta$; $M_{P,max} := \max(M_{P,max}, M_P)$
    endif
endprocedure

Алгоритм 3.2 – Зміна стану стохастичної мережі Петрі в момент t

```
procedure OutPhase(t)
    foreach randomly перехід T, для якого min E_T = t
        n := кількість моментів t у списку E_T;  вилучити їх зі списку
        foreach вихідна дуга (P, k) переходу T:  M_P := M_P + n·k
        if t > T_п then N_T := N_T + n
    endforeach
endprocedure

procedure InPhase(t)
    while існує перехід T, для якого X(M, T) = true do
        A := множина переходів з X(M, T) = true
        r* := найбільший пріоритет r_T серед переходів множини A
        обрати T серед переходів A з r_T = r* з імовірністю b_T / Σb
        foreach вхідна дуга (P, k) переходу T:  M_P := M_P − k
        додати момент t + g_T до списку E_T
        if кількість входів у момент t перевищує 10 000 then error
    endwhile
endprocedure
```

В алгоритмі $X(M, T)$ – умова активності переходу: у кожній вхідній позиції переходу кількість маркерів не менша за кратність дуги; N_T – лічильник виходів переходу на інтервалі спостереження; S_P, U_P – накопичені суми для середнього і середнього квадратичного відхилення маркування за формулами (2.3) і (2.4).

Порядок «спочатку всі виходи, потім входи» суттєвий для цієї моделі. Секція з'являється в позиції S в момент, кратний періоду конвеєра, і в той самий момент може завершитися надходження партії. Якщо рішення про секцію приймати одразу після виходу маркера з переходу «Рух конвеєра», до надходження партій цього моменту, секція буде пропущена, хоча вироби для неї є. Так, у сценарії з детермінованими інтервалами 5 і 20 хв у момент 20 хв одночасно надходять партії обох типів і подається секція: за прийнятого порядку секція бачить $Q_1 = 20$, $Q_2 = 20$ і заповнюється, а за послідовної обробки подій вона побачила б $Q_1 = 15$, $Q_2 = 0$ і була б пропущена. Вироби, не забрані цією секцією, залишилися б у накопичувачах до кінця прогону, і середні черги обох типів зросли б на 10 виробів. Порядок обробки всередині фази виходу на результат не впливає, оскільки виходи лише додають маркери.

Порівняно із загальним алгоритмом імітації стохастичної мережі Петрі [3] прийнято такі спрощення, допустимі для цієї моделі:

– мережа не містить інгібіторних дуг і маркерів різних типів, тому умова активності переходу зводиться до порівняння кількості маркерів з кратністю дуг;

– конфлікт за спільну вхідну позицію розв’язується у фазі входу вибором одного переходу за раз: перехід з меншим пріоритетом може спрацювати лише тоді, коли переходи з більшим пріоритетом неактивні;

– переходи надходження і руху конвеєра одноканальні завдяки петлям через позиції G_1, G_2 і C, а переходи з нульовою затримкою звільняються в той самий момент, тому список моментів виходу кожного переходу містить не більше одного елемента; багатоканальність збережена в загальному вигляді і не потребує окремих перевірок.

Щоб помилка в описі мережі (наприклад, цикл з переходів з нульовою затримкою) не призводила до нескінченного циклу, кількість входів в один момент часу обмежена 10 000; у разі перевищення імітація припиняється з повідомленням про помилку.

3.3 Структура програми

Програма складається з пакетів src/petri (алгоритм імітації стохастичної мережі Петрі, не пов’язаний з конкретною системою), src/model (мережа комплектувального конвеєра і консольний запуск моделі), src/experiments (верифікація та експериментальне дослідження) і tests (автоматизовані тести). Діаграму класів пакета src/petri і модуля мережі конвеєра наведено на рисунку 3.1, призначення класів – у таблиці 3.1.

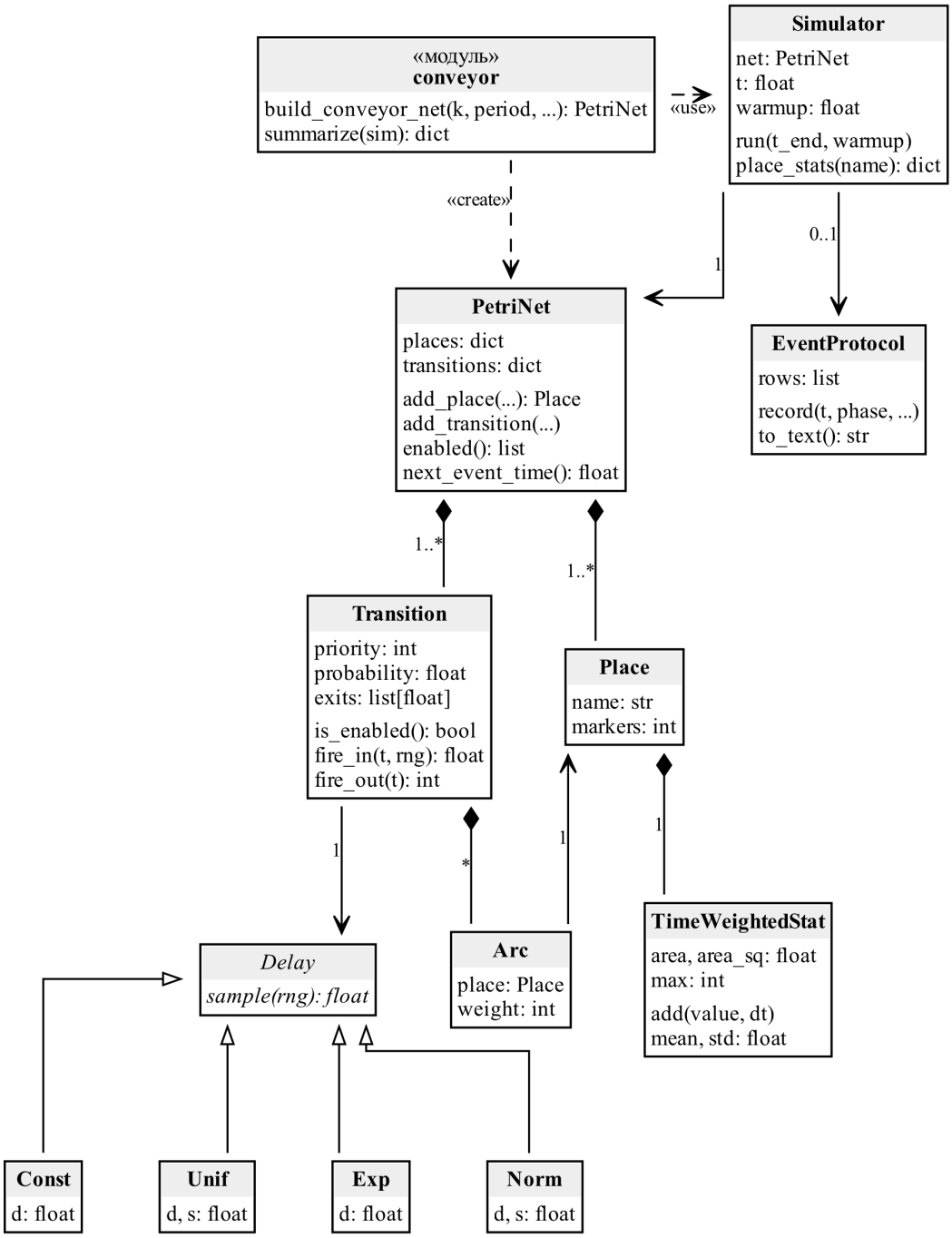


Рисунок 3.1 – Діаграма класів програми

Таблиця 3.1 – Класи та модулі програми

Клас або модуль	Призначення	Основні атрибути й методи
Place	позиція мережі	markers – кількість маркерів, stat – статистика маркування
Arc	дуга між позицією і переходом	place – позиція, weight – кратність
Transition	перехід з затримкою, пріоритетом і ймовірністю спрацювання	exits – список моментів виходу; is_enabled(), fire_in(t, rng), fire_out(t)
PetriNet	мережа: позиції, переходи, маркування	add_place(), add_transition(), enabled(), next_event_time()

Продовження таблиці 3.1

Клас або модуль	Призначення	Основні атрибути й методи
Simulator	алгоритм імітації (алгоритм 3.1)	run(t_end, warmup), _out_phase(), _in_phase(), _choose(), _settle(), _collect()
TimeWeightedStat	статистика величини, зваженої за часом	add(value, dt); mean, std, max
Delay, Const, Unif, Exp, Norm	детерміновані й випадкові затримки	sample(rng)
EventProtocol	протокол подій	record(), to_text()
conveyor (модуль)	мережа конвеєра і вихідні характеристики	build_conveyor_net(k, period, …, buffer), summarize(sim)

Класи Place, Arc, Transition і PetriNet описують структуру мережі, клас Simulator виконує алгоритм імітації над будь-якою мережею. Затримка переходу задається об’єктом класу, похідного від Delay, у позначеннях [3]: unif(5, 1) відповідає формулі (2.1). Функція build_conveyor_net будує мережу рисунка 2.1 з параметрами k і T_c, а з параметром buffer – мережу рисунка 2.2. Функція summarize обчислює вихідні характеристики за формулами (2.2)–(2.8).

Тести перевіряють закон збереження виробів (кількість виробів, що надійшли, дорівнює залишку в накопичувачі плюс $k \cdot N_{fill}$), кількість поданих секцій $N_{fill} + N_{skip} = \lfloor T_{\text{мод}} / T_c \rfloor$, відсутність пропуску секції за наявності виробів, збіг детермінованого сценарію з ручним розрахунком, нерівність $Q_i \le L_i$ у моделі з накопичувачем і відтворюваність результатів за однакового початкового значення генератора.

Вихідний код програми, результати експериментів і діаграми розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation-course-work. Модель запускається з командного рядка (python -m src.model.run) з параметрами k, T_c, розмірами партій, інтервалами надходження, часом моделювання, кількістю прогонів і місткостями накопичувачів; ключ --trace виводить протокол подій. Код алгоритму імітації наведено в додатку А.

3.4 Просування модельного часу та зміна стану мережі

Змінні стану алгоритму та їх програмну реалізацію наведено в таблиці 3.2.

Таблиця 3.2 – Змінні стану алгоритму імітації

Змінна	Позначення	Реалізація в програмі
Поточний модельний час, хв	t	Simulator.t
Маркування позиції	M_P	Place.markers
Моменти виходу маркерів з переходу, хв	E_T	Transition.exits
Кількість виходів переходу на інтервалі спостереження	N_T	Simulator.window_out
Тривалість перехідного періоду, хв	$T_{\text{п}}$	Simulator.warmup
Накопичені суми статистики маркування	S_P, U_P, $M_{P,max}$	TimeWeightedStat.area, area_sq, max

Головний цикл методу Simulator.run реалізує алгоритм 3.1. Вкладений цикл з викликом on_sample використовується лише під час дослідження перехідного періоду: він фіксує поточні середні значення в моменти, кратні заданому кроку, не змінюючи стану мережі.

```
self._settle()
if self.observer:
    self.observer(self)
while True:
    t_next = self.net.next_event_time()
    bound = min(t_next, t_end)
    while next_sample <= bound:
        self._collect(self.t, next_sample)
        self.t = next_sample
        on_sample(self)
        next_sample += sample_every
    if t_next > t_end:
        self._collect(self.t, t_end)
        self.t = float(t_end)
        break
    self._collect(self.t, t_next)
    self.t = t_next
    self._out_phase()
    self._settle()
    if self.observer:
        self.observer(self)
return self
```

Кожен перехід зберігає впорядкований список моментів виходу. Вхід маркерів вилучає маркери з вхідних позицій і додає до списку новий момент виходу, вихід звільняє всі канали, момент виходу яких настав:

```
def fire_in(self, t, rng):
    for arc in self.inputs:
        arc.place.markers -= arc.weight
    exit_time = t + float(self.delay(rng))
    insort(self.exits, exit_time)
    return exit_time

def fire_out(self, t):
    n = 0
    while self.exits and self.exits[0] <= t:
        self.exits.pop(0)
        n += 1
    for arc in self.outputs:
        arc.place.markers += arc.weight * n
    return n
```

Фаза виходу перемішує переходи, готові до виходу, і виконує їх вихід; фаза входу запускає активні переходи по одному, доки активних не залишиться. Метод _settle повторює обидві фази, доки в поточний момент є виходи переходів з нульовою затримкою, і контролює обмеження на кількість входів:

```
def _out_phase(self):
    ready = [tr for tr in self.net.transitions.values() if
    tr.next_exit <= self.t]
    self.rng.shuffle(ready)
    for tr in ready:
        n = tr.fire_out(self.t)
        self.fired_out[tr.name] += n
        if self.t > self.warmup or self.warmup == 0:
            self.window_out[tr.name] += n
        if self.protocol is not None:
            for _ in range(n):
                self.protocol.record(self.t, "OUT", tr, self.net)
    return bool(ready)
def _in_phase(self):
    fired = 0
    while fired <= MAX_FIRINGS_PER_INSTANT:
        enabled = self.net.enabled()
        if not enabled:
            return fired
        tr = self._choose(enabled)
        tr.fire_in(self.t, self.rng)
        self.fired_in[tr.name] += 1
        if self.protocol is not None:
            self.protocol.record(self.t, "IN", tr, self.net)
        fired += 1
```

```
    return fired

def _settle(self):
    fired = self._in_phase()
    while self.net.next_event_time() <= self.t:
        if fired > MAX_FIRINGS_PER_INSTANT:
            raise RuntimeError(f"net does not settle at
    t={self.t}")
        self._out_phase()
        fired += self._in_phase()
```

Конфлікт переходів розв’язує метод _choose: з активних переходів обираються ті, що мають найбільший пріоритет, а якщо таких кілька – один з них випадково з імовірностями, пропорційними *b*:

```
def _choose(self, enabled):
    top = max(tr.priority for tr in enabled)
    group = [tr for tr in enabled if tr.priority == top]
    if len(group) == 1:
        return group[0]
    weights = [tr.probability for tr in group]
    total = sum(weights)
    idx = self.rng.choice(len(group), p=[w / total for w in
    weights])
    return group[idx]
```

У мережі конвеєра пріоритети задано під час створення переходів. «Комплектація» має пріоритет 1, тому «Пропуск секції» обирається лише тоді, коли «Комплектація» неактивна, тобто виробів хоча б одного типу менше за *k*:

```
net.add_transition(FILL, const(0), {"S": 1, "Q1": k, "Q2": k},
                   fill_out, priority=1)
net.add_transition(SKIP, const(0), {"S": 1}, {"N_skip": 1},
    priority=0)
```

3.5 Обчислення вихідних характеристик

Маркування між двома послідовними подіями не змінюється, тому статистика накопичується один раз на інтервал: до сум додаються добутки значення на тривалість інтервалу. Значення, що існують нульовий час (усередині

одного моменту між фазами), до середнього і максимуму не потрапляють, що відповідає формулам (2.3)–(2.5):

```
def add(self, value, dt):
    if dt <= 0:
        return
    self.total_time += dt
    self.area += value * dt
    self.area_sq += value * value * dt
    self.max = value if self.max is None else max(self.max, value)
    self.min = value if self.min is None else min(self.min, value)

def _collect(self, t_from, t_to):
    lo = max(t_from, self.warmup)
    dt = t_to - lo
    if dt <= 0:
        return
    for place in self.net.places.values():
        place.stat.add(place.markers, dt)
```

Статистика збирається для всіх позицій мережі, тому характеристики позицій Q_1, Q_2 і G_i обчислюються однаково. Лічильники window_out враховують виходи переходів у моменти після $T_п$. За ними функція summarize обчислює ймовірність пропуску секції, продуктивність і фактичну інтенсивність надходження:

```
params = sim.net.params
n_fill = sim.window_out[FILL]
n_skip = sim.window_out[SKIP]
total = n_fill + n_skip
span = sim.t - sim.warmup
q1 = sim.place_stats("Q1")
q2 = sim.place_stats("Q2")
lam1 = params["batch1"] * sim.window_out[ARRIVAL_1] / span
lam2 = params["batch2"] * sim.window_out[ARRIVAL_2] / span
```

Один прогін дає одне значення кожної вихідної характеристики. Оцінкою характеристики є середнє за p незалежними прогонами, точність оцінки задає півширина довірчого інтервалу з довірчою ймовірністю β = 0,95 за розподілом Стьюдента:

$$\delta = t_{\beta;p-1} \cdot \frac{s}{\sqrt{p}} \tag{3.1}$$

де δ – півширина довірчого інтервалу середнього значення;

$t_{\beta;\,p-1}$ – квантиль рівня $(1 + \beta) / 2$ розподілу Стьюдента з $p - 1$ ступенями вільності;

s – вибіркове середнє квадратичне відхилення характеристики за p прогонами;

p – кількість прогонів.

Далі результати подаються у вигляді «середнє ± δ». Кожен прогін використовує власне початкове значення генератора, тому прогони незалежні, а результати відтворювані.

3.6 Верифікація моделі

Верифікація полягає в тому, щоб для набору змінених вхідних даних отримати результати, які можна передбачити заздалегідь, і порівняти їх з результатами моделювання. Очікувані значення обчислено аналітично: за середніми інтенсивностями надходження і споживання виробів, за ручним розрахунком для детермінованих інтервалів і за оцінками теорії відновлення, наведеними в підрозділі 4.2.

У кожному рядку змінено один або два параметри, решта значень відповідає базовій моделі: $k = 10$, $T_c = 10$ хв, партії по 5 виробів типу 1 з інтервалом unif(5, 1) хв і по 20 виробів типу 2 з інтервалом unif(20, 7) хв, порожні накопичувачі в початковий момент, $T_{\text{п}} = 0$. Час моделювання $T_{\text{мод}} = 2000$ хв, кількість прогонів 100 (у детермінованому сценарії – 1 прогін), після знака «±» наведено півширину 95 % довірчого інтервалу за формулою (3.1). Швидкість зростання черги оцінено як $Q_i(T_{\text{мод}}) / T_{\text{мод}}$. Результати наведено в таблиці 3.3.

Таблиця 3.3 – Результати верифікації моделі

№	Зміна вхідних даних	Характеристика	Отримане значення	Очікуване значення
1	без змін	P_{skip}	0,0227 ± 0,0026	$\to 0$ при $T_{\text{мод}} \to \infty$
		$\bar{Q}_1 - \bar{Q}_2$, од.	7,10 ± 4,89	7,13
		$П$, виробів/год	58,64 ± 0,15	$60 \cdot (1 - P_{skip}) =$ 58,64
2	інтервали 5 і 20 хв (детерміновані)	P_{skip}	0,0050	0,0050
		$\bar{Q}_1$, од.	12,45	12,45
		$\bar{Q}_2$, од.	4,95	4,95
		$Q_{1,max}$, од.	15	15

Продовження таблиці 3.3

№	Зміна вхідних даних	Характеристика	Отримане значення	Очікуване значення
2	інтервали 5 і 20 хв (детерміновані)	$Q_{2,max}$, од.	10	10
3	інтервал типу 2 unif(40, 7) хв	P_{skip}	0,5056 ± 0,0015	0,5
		Швидкість зростання Q_1, од./хв	0,504 ± 0,002	0,5
		$\bar{Q}_1 - \bar{Q}_2$, од.	506,8 ± 2,1	507,4
		$Q_{2,max}$, од.	20	20
		$П$, виробів/год	29,66 ± 0,09	30
4	T_c = 5 хв	P_{skip}	0,5070 ± 0,0012	0,5
		$П$, виробів/год	59,17 ± 0,14	60
		Приріст $\bar{Q}_1 + \bar{Q}_2$ при збільшенні $T_{мод}$ до 10 000 хв, од.	27,9 ± 5,1	27,6
5	T_c = 20 хв	P_{skip}	0,0054 ± 0,0010	0,5 / 100 = 0,005
		Швидкість зростання Q_1, од./хв	0,501 ± 0,001	0,5
		Швидкість зростання Q_2, од./хв	0,498 ± 0,004	0,5
		$\bar{Q}_1 - \bar{Q}_2$, од.	6,58 ± 4,46	7,13
		$П$, виробів/год	29,84 ± 0,03	30
6	k = 20, T_c = 20 хв	P_{skip}	0,0209 ± 0,0022	$\to 0$ при $T_{мод} \to \infty$
		$\bar{Q}_1 - \bar{Q}_2$, од.	8,58 ± 4,39	7,13
		$П$, виробів/год	58,75 ± 0,13	$60 \cdot (1 - P_{skip})$ = 58,75
7	b_1 = 10	P_{skip}	0,0223 ± 0,0026	як у рядку 1: 0,0227
		Швидкість зростання Q_1, од./хв	1,019 ± 0,003	1,0
		$\bar{Q}_1 - \bar{Q}_2$, од.	1004,1 ± 5,5	1004,7

Коментарі до рядків таблиці:

Рядок 1. Інтенсивності надходження обох типів і найбільша швидкість комплектації дорівнюють 1 виробу за хвилину, тому за ρ = 1 ймовірність пропуску прямує до нуля лише за необмеженого збільшення часу моделювання (підрозділ 4.2); за скінченного $T_{мод}$ вона мала, але додатна. Різниця середніх черг збігається з оцінкою теорії відновлення в межах довірчого інтервалу. Продуктивність точно дорівнює $60 \cdot (1 - P_{skip})$: кожна заповнена секція забирає рівно k виробів кожного типу.

Рядок 2. Ручний розрахунок: перша партія типу 2 надходить у момент 20 хв, тому секція в момент 10 хв пропускається ($Q_1 = 10$, $Q_2 = 0$), а всі наступні

заповнюються; P_{skip} = 1 / 200. З моменту 20 хв черга типу 1 по 5 хв дорівнює 10 і 15 виробам, черга типу 2 по 10 хв – 10 і 0 виробам, до моменту 20 хв черга типу 1 зростає ступенями 0, 5, 10, 15, а черга типу 2 порожня. Звідси $\bar{Q}_1$ = (150 + 12,5·(2000 − 20)) / 2000 = 12,45, $\bar{Q}_2$ = 5·(2000 − 20) / 2000 = 4,95. Усі п’ять характеристик збігаються з розрахунком точно.

Рядок 3. Інтенсивність надходження виробів типу 2 зменшується до 0,5 виробу/хв, і саме вона обмежує комплектацію: заповнюється в середньому половина секцій, а вироби типу 1 накопичуються зі швидкістю 1 − 0,5 = 0,5 виробу/хв. Черга типу 2 не перевищує однієї партії, бо кожну партію забирають дві найближчі секції. Незначне перевищення P_{skip} над 0,5 пояснюється скінченністю $T_{\text{мод}}$: перша партія типу 2 надходить у середньому лише через 40 хв, а вироби останньої партії до кінця прогону забрано не повністю.

Рядок 4. Конвеєр подає секції вдвічі частіше, ніж їх можна заповнити, тому пропускається половина секцій, а продуктивність обмежена надходженням, 60 виробів/год. Можна було б очікувати, що за такого запасу швидкості черги стануть обмеженими, однак модель показує інше: сума середніх черг зростає з 34,0 виробу за $T_{\text{мод}}$ = 2000 хв до 62,0 виробу за $T_{\text{мод}}$ = 10 000 хв (100 прогонів). Причина структурна: кожна заповнена секція забирає однакову кількість виробів обох типів, тому різниця черг $Q_1 - Q_2$ дорівнює різниці кількостей виробів, що надійшли, за будь-якої швидкості конвеєра. За рівних середніх інтенсивностей ця різниця є випадковим блуканням без зносу, і надлишок одного з типів накопичується пропорційно $\sqrt{T_{\text{мод}}}$. Швидкий конвеєр лише утримує меншу з двох черг біля нуля. Оцінка приросту за формулою, виведеною в підрозділі 4.2, збігається з результатом моделювання, отже, модель правильно відтворює і цю властивість системи.

Рядок 5. Конвеєр може забрати лише 0,5 виробу кожного типу за хвилину, тому обидві черги зростають з цією швидкістю, а продуктивність становить 30 виробів/год. Секцію може бути пропущено лише першу (у момент 20 хв): вона порожня, якщо перша партія типу 2 ще не надійшла, а інтервал unif(20, 7)

симетричний відносно 20 хв, тому ймовірність цього 0,5. На 100 секцій це дає 0,005.

Рядок 6. Варіант по 20 виробів має ті самі середні інтенсивності надходження і ту саму найбільшу швидкість комплектації 20 / 20 = 1 виріб/хв, тому якісно поводиться так само, як базова модель: мала ймовірність пропуску, різниця черг за теорією відновлення, продуктивність $60\cdot(1 - P_{skip})$.

Рядок 7. Надходження виробів типу 1 подвоюється, комплектацію обмежують вироби типу 2. Черга типу 1 зростає зі швидкістю 2 − 1 = 1 виріб/хв, а ймовірність пропуску в межах довірчого інтервалу дорівнює значенню рядка 1: секції пропускаються через нестачу виробів типу 2, інтервали надходження яких не змінилися.

В усіх рядках отримані значення збігаються з очікуваними в межах довірчих інтервалів або відрізняються на величини, причину яких пояснено. Отже, алгоритм імітації і модель відтворюють поведінку системи правильно.

Покрокову роботу алгоритму перевірено за протоколом подій детермінованого сценарію рядка 2. Протокол фіксує кожну фазу входу і виходу маркерів і маркування позицій після неї. Фрагмент протоколу за перші 40 хв наведено в таблиці 3.4, повний протокол за $T_{мод}$ = 2000 хв містить 1803 подій, його продовження до 80 хв наведено в додатку Д.

Таблиця 3.4 – Фрагмент протоколу подій детермінованого сценарію

t, хв	Фаза	Перехід	Q_1	Q_2	S	N_{fill}	N_{skip}
0	вхід	Надходження 2	0	0	0	0	0
0	вхід	Рух конвеєра	0	0	0	0	0
0	вхід	Надходження 1	0	0	0	0	0
5	вихід	Надходження 1	5	0	0	0	0
5	вхід	Надходження 1	5	0	0	0	0
10	вихід	Надходження 1	10	0	0	0	0
10	вихід	Рух конвеєра	10	0	1	0	0
10	вхід	Пропуск секції	10	0	0	0	0
10	вхід	Надходження 1	10	0	0	0	0
10	вхід	Рух конвеєра	10	0	0	0	0
10	вихід	Пропуск секції	10	0	0	0	1
15	вихід	Надходження 1	15	0	0	0	1
15	вхід	Надходження 1	15	0	0	0	1
20	вихід	Надходження 2	15	20	0	0	1
20	вихід	Надходження 1	20	20	0	0	1
20	вихід	Рух конвеєра	20	20	1	0	1

Продовження таблиці 3.4

t, хв	Фаза	Перехід	Q_1	Q_2	S	N_{fill}	N_{skip}
20	вхід	Комплектація	10	10	0	0	1
20	вхід	Рух конвеєра	10	10	0	0	1
20	вхід	Надходження 1	10	10	0	0	1
20	вхід	Надходження 2	10	10	0	0	1
20	вихід	Комплектація	10	10	0	1	1
25	вихід	Надходження 1	15	10	0	1	1
25	вхід	Надходження 1	15	10	0	1	1
30	вихід	Надходження 1	20	10	0	1	1
30	вихід	Рух конвеєра	20	10	1	1	1
30	вхід	Комплектація	10	0	0	1	1
30	вхід	Рух конвеєра	10	0	0	1	1
30	вхід	Надходження 1	10	0	0	1	1
30	вихід	Комплектація	10	0	0	2	1
35	вихід	Надходження 1	15	0	0	2	1
35	вхід	Надходження 1	15	0	0	2	1
40	вихід	Надходження 2	15	20	0	2	1
40	вихід	Надходження 1	20	20	0	2	1
40	вихід	Рух конвеєра	20	20	1	2	1
40	вхід	Комплектація	10	10	0	2	1
40	вхід	Рух конвеєра	10	10	0	2	1
40	вхід	Надходження 2	10	10	0	2	1
40	вхід	Надходження 1	10	10	0	2	1
40	вихід	Комплектація	10	10	0	3	1

У момент 0 маркери входять у переходи надходження і руху конвеєра, тобто плануються перша партія кожного типу і перша секція. У момент 10 хв спочатку виходять маркери з переходів «Надходження 1» і «Рух конвеєра», а потім, оскільки $Q_2 = 0$, спрацьовує «Пропуск секції». У момент 20 хв три виходи (партії обох типів і секція) виконуються до будь-якого входу, і лише після цього «Комплектація» забирає по 10 виробів кожного типу. Вихід переходу «Комплектація» з нульовою затримкою з’являється в протоколі в той самий момент після входів, що підтверджує повторення фаз у межах одного моменту часу. Код верифікації наведено в додатку Б.

3.7 Висновок до розділу

Розроблено власний алгоритм імітації стохастичної мережі Петрі з багатоканальними переходами, пріоритетами і ймовірностями спрацювання та реалізовано його мовою Python. Час просувається за найближчою подією; у

кожний момент спочатку виконуються всі виходи маркерів, потім входи, і фази повторюються, доки є виходи переходів з нульовою затримкою. Такий порядок гарантує, що рішення про секцію враховує всі вироби, що надійшли в момент її подачі. Статистика маркування збирається зважено за часом. Верифікація на 7 наборах вхідних даних підтвердила правильність алгоритму і моделі та виявила, що різниця черг двох типів не залежить від швидкості конвеєра, що враховано під час планування експериментів.

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ

4.1 Мета експерименту, фактор і відгуки

Експериментальне дослідження має дві мети, що випливають із завдання: отримати статистичні оцінки ймовірності пропуску секції, середніх і максимальних черг у базовому варіанті та з'ясувати, чи впливає перехід на комплектацію по 20 виробів на ці характеристики. Третя мета – обґрунтувати параметри модифікації з обмеженим накопичувачем, яку запропоновано в розділі 5.

Відгуками моделі є ймовірність пропуску секції P_{skip}, середні довжини черг $\bar{Q}_1$, $\bar{Q}_2$, максимальні довжини черг $Q_{1,max}$, $Q_{2,max}$ і продуктивність $П$. Для порівняння варіантів фактором є схема комплектації – якісний фактор з двома рівнями: 10 виробів у секції і секція кожні 10 хв (10/10) та 20 виробів і секція кожні 20 хв (20/20). Для моделі з накопичувачем факторами є місткості L_1 і L_2. Серії експериментів наведено в таблиці 4.1.

Таблиця 4.1 – Серії експериментів

Серія	Модель	Фактор і його рівні	$T_{мод}$, хв	Кількість прогонів
Статистичні оцінки	базова	–	480; 2400	1600
Дисперсійний аналіз	базова, 20/20	схема комплектації: 10/10, 20/20	480; 2400	1600 на рівень
Регресійний аналіз і пошук	з накопичувачем	L_1 = 15, 20, 25, 30; L_2 = 30, 40, …, 80	40 000 ($T_п$ = 3000)	120 на точку

Часи моделювання і кількості прогонів обґрунтовано в підрозділах 4.3, 4.4 і 4.7. Прогони різних рівнів фактора використовують різні діапазони початкових значень генератора, тому вибірки рівнів незалежні: вибірковий коефіцієнт кореляції між прогонами з однаковим номером не перевищує за модулем 0,03.

4.2 Теоретичний розрахунок характеристик системи

Середня інтенсивність надходження виробів типу i дорівнює відношенню розміру партії до середнього інтервалу між партіями:

$$\lambda_i = \frac{b_i}{\overline{\tau_i}} \tag{4.1}$$

де $\overline{\tau_i}$ – середній інтервал між партіями типу i, хв.

Для базових параметрів $\lambda_1 = 5 / 5 = 1$ виріб/хв, $\lambda_2 = 20 / 20 = 1$ виріб/хв. Конвеєр може укомплектувати не більше k / T_c виробів кожного типу за хвилину, тому коефіцієнт завантаження

$$\rho = \frac{\max(\lambda_1, \lambda_2)}{k/T_c} \tag{4.2}$$

дорівнює 1 як у базовому варіанті (10 / 10), так і у варіанті 20/20 (20 / 20). Якщо нехтувати випадковістю, частка заповнених секцій визначається найменшою з інтенсивностей, і ймовірність пропуску становить

$$P_{skip} \approx 1 - \frac{\min(\lambda_1, \lambda_2, k/T_c)}{k/T_c} \tag{4.3}$$

За $\rho = 1$ формула (4.3) дає $P_{skip} \approx 0$, а черги в середньому не зростають. Ця оцінка описує лише граничну поведінку: за скінченного часу секції пропускаються через випадкові коливання надходжень, а самі черги, як показано нижче, не мають сталого режиму. Для рядків 3–5 таблиці 3.3 формула (4.3) дає $P_{skip} \approx 0{,}5$, 0,5 і 0 відповідно, що використано під час верифікації.

Кожна заповнена секція забирає однакову кількість k виробів обох типів, а пропущена не забирає нічого. Тому за порожніх у початковий момент накопичувачів у будь-який момент часу виконується рівність

$$Q_1(t) - Q_2(t) = A_1(t) - A_2(t) \tag{4.4}$$

де $A_i(t)$ – кількість виробів типу i, що надійшли на інтервалі $[0; t]$.

Рівність не залежить ні від швидкості конвеєра, ні від k. Математичне сподівання кількості виробів, що надійшли, за теорією відновлення [8, с. 439–518] для великих t дорівнює

$$\mathrm{E}\big(A_i(t)\big) \approx \lambda_i \cdot t + \frac{b_i\big({\sigma_i}^2 - {\overline{\tau_i}}^2\big)}{2{\overline{\tau_i}}^2} \tag{4.5}$$

де σ_i – середнє квадратичне відхилення інтервалу між партіями типу i, хв; для unif(d, s) $\sigma^2 = (2s)^2 / 12$.

Другий доданок дорівнює −2,47 для виробів типу 1 і −9,59 для виробів типу 2: перша партія типу 2 надходить лише через 20 хв у середньому і одразу приносить 20 виробів. Тому за рівних інтенсивностей середня черга типу 1 більша за середню чергу типу 2 на сталу величину 7,13 виробу, яка не зростає з часом. Перевірку цієї оцінки наведено в таблиці 4.2.

Таблиця 4.2 – Різниця середніх черг і оцінка теорії відновлення

Надходження виробів	$T_{\text{мод}}$, хв	Кількість прогонів	$\bar{Q}_1 - \bar{Q}_2$ за моделлю, од.	Оцінка за формулою (4.5), од.
Базовий: 5 шт. unif(5, 1) / 20 шт. unif(20, 7)	10 000	200	9,3 ± 7,3	7,13
Обидва типи: 5 шт. unif(5, 1)	10 000	200	0,5 ± 3,2	0,00
Тип 2: 20 шт. unif(20, 1)	10 000	200	6,7 ± 2,2	7,53

У другому рядку обидва типи надходять однаковими партіями по 5 виробів, у третьому інтервал типу 2 майже детермінований, решта параметрів відповідає базовій моделі. В усіх випадках оцінка лежить у межах довірчого інтервалу.

Важливіший наслідок рівності (4.4) стосується випадкової частини. За $\lambda_1 = \lambda_2$ різниця $A_1 - A_2$ є випадковим блуканням без зносу, дисперсія якого зростає пропорційно часу зі швидкістю

$$D = \sum_{i=1}^{2} \frac{{b_i}^2 {\sigma_i}^2}{{\overline{\tau_i}}^3} \tag{4.6}$$

Для базових параметрів D = 0,883 виробу² за хвилину, з них 8 % припадає на вироби типу 1, решта – на вироби типу 2. Оскільки обидві черги невід’ємні, черга того типу, що надійшов у більшій кількості, не менша за $|A_1 - A_2|$. Наближаючи цю різницю нормальним розподілом, отримуємо середнє за інтервал [0; $T_{\text{мод}}$] значення надлишку

$$\overline{H} \approx \frac{2}{3}\sqrt{\frac{2D}{\pi}} \cdot \sqrt{T_{\text{мод}}} = 0{,}50 \cdot \sqrt{T_{\text{мод}}} \qquad (4.7)$$

де $\bar{H}$ – середня за часом кількість виробів одного типу, яка перевищує кількість виробів іншого типу, од.

Для $T_{\text{мод}}$ = 480 хв це 11,0 виробу, для 2400 хв – 24,5, для 40 000 хв – 100. Комплектація не може зменшити цей надлишок, бо забирає вироби обох типів порівну. Отже, середні й максимальні черги базової системи зростають необмежено за будь-якої швидкості конвеєра; швидший конвеєр лише утримує меншу з черг поблизу нуля і збільшує частку пропущених секцій (рядок 4 таблиці 3.3). Ймовірність пропуску і продуктивність, навпаки, мають границі: $P_{skip} \to 0$ і $П \to 60$ виробів/год за $T_{\text{мод}} \to \infty$.

Звідси випливає план дослідження. Для черг базової системи і варіанта 20/20 сталого режиму не існує, тому їх оцінюють для заданого скінченного часу роботи. Модель, у якій сталий режим існує, можна отримати лише змінивши систему так, щоб надлишок одного типу не накопичувався, – обмеживши місткість накопичувачів.

4.3 Перехідний період і вибір часу моделювання

Тривалість перехідного періоду визначається за кількома прогонами моделі як момент T, після якого поточні середні значення відгуку в будь-яких двох прогонах відрізняються менше ніж на задану точність ε [3]:

$$\left|\overline{y}_i(T) - \overline{y}_j(T)\right| < \varepsilon, \quad \forall i, j \qquad (4.8)$$

де $\bar{y}_i(T)$ – середнє значення відгуку в i-му прогоні, обчислене за формулою (2.3) на інтервалі [0; T];

ε – задана точність.

Для базового варіанта і варіанта 20/20 виконано по 5 прогонів тривалістю 20 000 хв. Поточні середні значення відгуків базового варіанта наведено на рисунку 4.1.

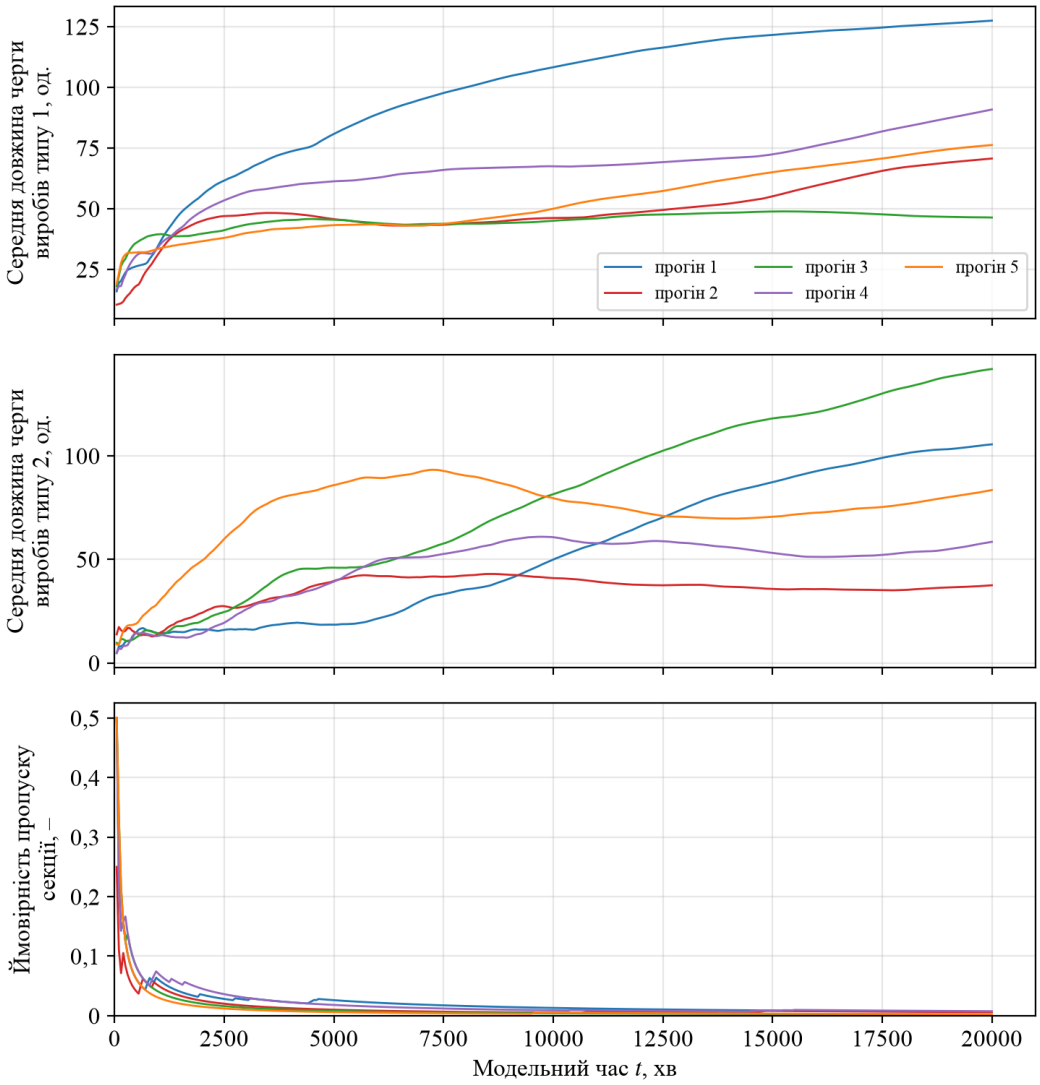


Рисунок 4.1 – Поточні середні значення відгуків базового варіанта, 5 прогонів

Розкид поточних середніх між прогонами (найбільша різниця за модулем) і середні за п’ятьма прогонами значення для обох варіантів наведено в таблиці 4.3.

Таблиця 4.3 – Розкид поточних середніх значень за 5 прогонами

Варіант	T, хв	Середнє $\bar{Q}_1$, од.	Розкид $\bar{Q}_1$, од.	Середнє $\bar{Q}_2$, од.	Розкид $\bar{Q}_2$, од.	Середнє P_{skip}	Розкид P_{skip}
10/10	480	28,6	17,7	15,0	6,5	0,0653	0,0408
10/10	2400	47,7	23,0	28,6	42,0	0,0234	0,0251
10/10	5000	55,3	37,6	45,8	67,3	0,0136	0,0200
10/10	10 000	63,4	63,2	62,5	40,5	0,0072	0,0090
10/10	20 000	82,3	81,1	85,3	104,4	0,0049	0,0055
20/20	480	33,4	16,3	18,7	12,6	0,0667	0,0417
20/20	2400	46,2	28,8	36,8	36,1	0,0218	0,0336
20/20	5000	53,8	31,1	53,8	69,0	0,0112	0,0161
20/20	10 000	63,6	58,0	79,9	107,0	0,0072	0,0080
20/20	20 000	75,8	64,2	92,3	137,2	0,0038	0,0040

Для варіанта 20/20 характер кривих такий самий (рис. 4.2).

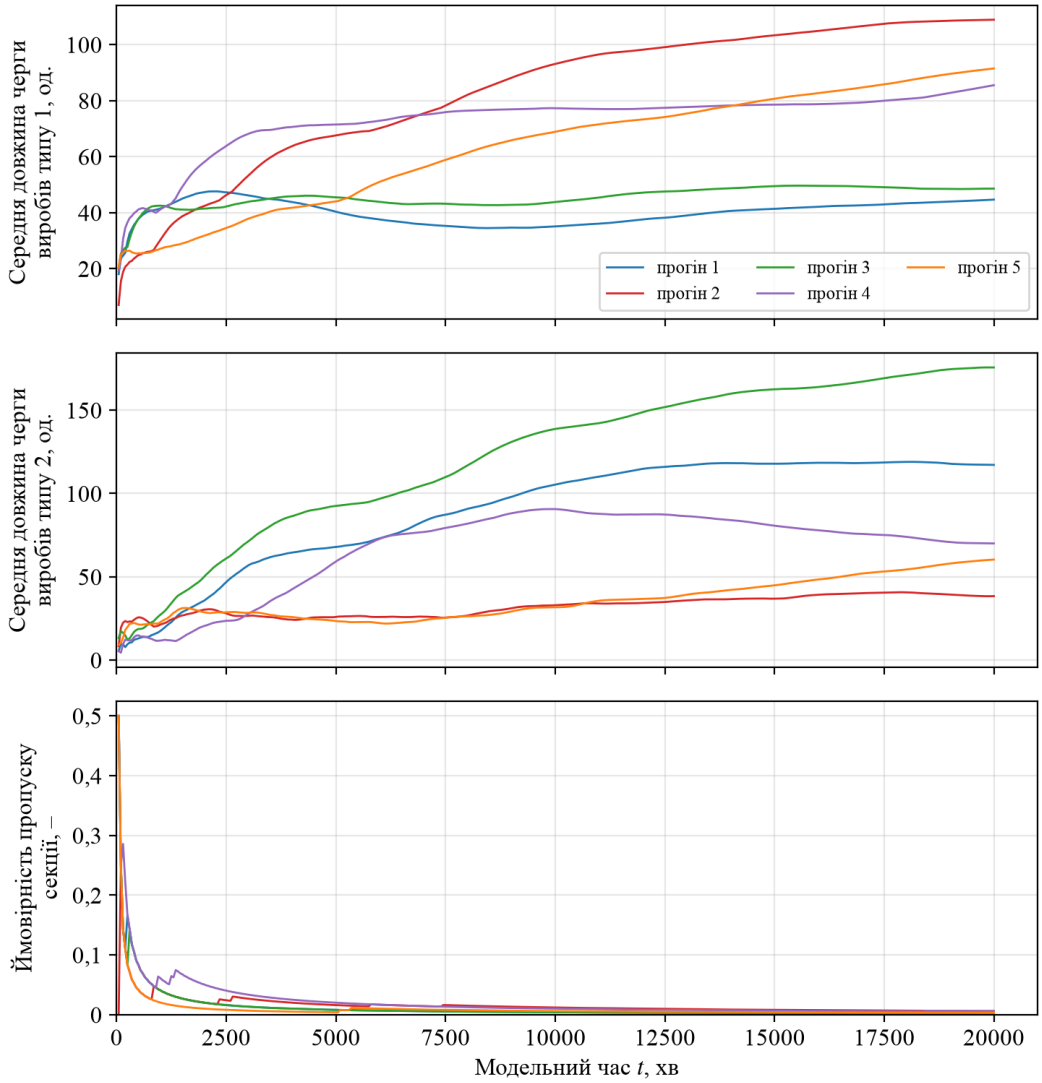


Рисунок 4.2 – Поточні середні значення відгуків варіанта 20/20, 5 прогонів

Розкид середніх черг не зменшується з часом, а зростає: для базового варіанта – з 17,7 до 81,1 виробу для $\bar{Q}_1$ і з 6,5 до 104,4 для $\bar{Q}_2$ між T = 480 і 20 000 хв. Отже, умова (4.8) для черг не виконується за жодного розумного ε, що узгоджується з підрозділом 4.2. Розкид P_{skip} зменшується, але саме середнє значення продовжує спадати (з 0,065 до 0,0049), тобто сталого значення P_{skip} також не досягає.

Тому для базового варіанта і варіанта 20/20 застосовано моделювання на скінченному інтервалі. Система моделюється від початку робочої зміни, коли накопичувачі порожні, протягом $T_{\text{мод}}$ = 480 хв (одна зміна); статистика збирається з нульового моменту, оскільки початковий стан тут є частиною

досліджуваного режиму. Для оцінки того, як результати залежать від тривалості безперервної роботи, додатково використано $T_{мод}$ = 2400 хв (п’ять змін без спорожнення накопичувачів). Усі результати для цих моделей наведено із зазначенням $T_{мод}$.

4.4 Визначення кількості прогонів

Кількість прогонів, що забезпечує точність ε з довірчою ймовірністю β, визначається за наслідком нерівності Чебишева [3]:

$$p = \frac{\sigma^2}{\varepsilon^2(1-\beta)} = 20 \cdot \left(\frac{\sigma}{\varepsilon}\right)^2, \ \beta = 0{,}95 \qquad (4.9)$$

де σ – середнє квадратичне відхилення відгуку між прогонами;

ε – задана точність оцінки.

Точність задано з огляду на практичний зміст відгуків: ε = 0,005 для P_{skip} (≈ 0,24 секції за зміну з 48), ε = 1 виріб для середніх черг і ε = 2 вироби для максимальних. σ оцінено за 100 пробними прогонами, початкові значення генератора яких не збігаються з основною серією. Результати розрахунку наведено в таблиці 4.4.

Таблиця 4.4 – Кількість прогонів за формулою (4.9)

$T_{мод}$, хв	Відгук	ε	σ, 10/10	p, 10/10	σ, 20/20	p, 20/20
480	P_{skip}	0,005	0,0248	491	0,0232	430
	$\bar{Q}_1$, од.	1	8,12	1319	6,00	722
	$\bar{Q}_2$, од.	1	5,38	578	8,77	1540
	$Q_{1,max}$, од.	2	12,36	764	10,94	599
	$Q_{2,max}$, од.	2	9,24	428	14,09	994
2400	P_{skip}	0,005	0,0111	99	0,0118	111
	$\bar{Q}_1$, од.	1	19,17	7350	17,34	6014
	$\bar{Q}_2$, од.	1	14,06	3957	13,57	3681
	$Q_{1,max}$, од.	2	27,57	3802	28,29	4001
	$Q_{2,max}$, од.	2	24,61	3029	23,52	2767

Для $T_{мод}$ = 480 хв найбільша необхідна кількість прогонів 1540 (відгук $\bar{Q}_2$ у варіанті 20/20), тому для обох рівнів фактора прийнято p = 1600. Для $T_{мод}$ = 2400 хв розкид черг значно більший і формула (4.9) вимагає до 7350 прогонів; ця серія також виконана з 1600 прогонами, тому її результати мають меншу точність і використовуються як допоміжні. Формула (4.9) спирається на нерівність

Чебишева і дає запас: фактична півширина 95 % довірчого інтервалу за формулою (3.1) для $\bar{Q}_1$ за $T_{мод}$ = 480 хв і 1600 прогонів становить 0,40 виробу, а для 2400 хв – 0,83 виробу.

4.5 Статистичні оцінки характеристик базової моделі

Статистичні оцінки відгуків базової моделі за 1600 незалежними прогонами наведено в таблиці 4.5. Для кожного відгуку вказано середнє за прогонами, середнє квадратичне відхилення між прогонами, межі 95 % довірчого інтервалу середнього і найменше та найбільше значення серед прогонів.

Таблиця 4.5 – Статистичні оцінки відгуків базової моделі, 1600 прогонів

$T_{мод}$, хв	Відгук	Середнє	СКВ	95 % довірчий інтервал	Мінімум	Максимум
480	P_{skip}	0,0589	0,0246	0,0577 … 0,0601	0,0208	0,1667
	$\bar{Q}_1$, од.	24,75	8,08	24,36 … 25,15	6,36	52,18
	$\bar{Q}_2$, од.	17,48	6,40	17,16 … 17,79	7,52	53,82
	$Q_{1,max}$, од.	38,56	12,02	37,97 … 39,15	15	85
	$Q_{2,max}$, од.	40,53	11,00	39,99 … 41,07	20	90
	$П$, виробів/год	56,46	1,48	56,39 … 56,54	50,00	58,75
2400	P_{skip}	0,0200	0,0106	0,0195 … 0,0206	0,0042	0,0750
	$\bar{Q}_1$, од.	38,53	16,96	37,70 … 39,36	7,39	119,05
	$\bar{Q}_2$, од.	31,50	14,43	30,80 … 32,21	11,04	107,36
	$Q_{1,max}$, од.	61,97	25,88	60,70 … 63,24	20	190
	$Q_{2,max}$, од.	71,84	23,88	70,67 … 73,01	30	190
	$П$, виробів/год	58,80	0,64	58,77 … 58,83	55,50	59,75

Відповідь на питання завдання для базового варіанта за $T_{мод}$ = 480 хв (1600 прогонів) така:

– ймовірність пропуску секції 0,0589 ± 0,0012, тобто в середньому 2,8 порожньої секції з 48 за зміну; в окремих змінах – від 1 до 8 порожніх секцій;

– середня черга виробів типу 1 24,75 ± 0,40 виробу, типу 2 – 17,48 ± 0,31 виробу;

– максимальна за зміну черга виробів типу 1 в середньому 38,6 ± 0,6 виробу, типу 2 – 40,5 ± 0,5 виробу; найбільші значення серед усіх прогонів – 85 і 90 виробів;

– продуктивність 56,46 ± 0,07 виробу кожного типу за годину.

Черга типу 1 у середньому довша за чергу типу 2 на 7,3 виробу, що близько до оцінки 7,1 з підрозділу 4.2. Великий розкид між прогонами (коефіцієнт варіації P_{skip} 42 %) означає, що окрема зміна може суттєво відрізнятися від середньої. За $T_{\text{мод}}$ = 2400 хв частка пропущених секцій зменшується до 0,0200, а середні і максимальні черги зростають: 38,5 і 31,5 виробу в середньому, 62,0 і 71,8 в максимумі. Це прояв відсутності сталого режиму: результат завдання для базової системи має сенс лише разом із часом моделювання.

4.6 Дисперсійний аналіз варіантів комплектації

Вплив схеми комплектації на кожен відгук перевірено однофакторним дисперсійним аналізом [3]. Фактор має q = 2 рівні, на кожному рівні виконано p = 1600 прогонів. Факторна дисперсія характеризує відмінність середніх за рівнями, залишкова – розкид прогонів усередині рівнів:

$$D_{\text{ф}} = \frac{p \cdot \sum_{j=1}^{q} \left(\overline{y}_j - \overline{y}\right)^2}{q - 1} \tag{4.10}$$

$$D_{\text{з}} = \frac{\sum_{j=1}^{q} \sum_{s=1}^{p} \left(y_{js} - \overline{y}_j\right)^2}{q(p - 1)} \tag{4.11}$$

де y_{js} – значення відгуку в s-му прогоні на j-му рівні фактора;

$\bar{y}_j$ – середнє значення відгуку на j-му рівні;

$\bar{y}$ – загальне середнє значення відгуку.

Вплив фактора вважається значущим, якщо

$$F = \frac{D_{\text{ф}}}{D_{\text{з}}} > F_{\text{кр}}\left(\alpha; q - 1; q(p - 1)\right) \tag{4.12}$$

де $F_{\text{кр}}$ – критичне значення розподілу Фішера для рівня значущості α = 0,05.

У формулі (4.10) суму квадратів поділено на кількість ступенів вільності q − 1, як у стандартному однофакторному дисперсійному аналізі; за такого знаменника відношення (4.12) має розподіл Фішера. Для q = 2 і p = 1600 критичне

значення $F_{кр}(0{,}05;\ 1;\ 3198) = 3{,}84$. Власна реалізація дає ті самі значення F, що й функція f_oneway бібліотеки SciPy [6]. Результати наведено в таблиці 4.6.

Таблиця 4.6 – Дисперсійний аналіз схем комплектації, 1600 прогонів на рівень

$T_{мод}$, хв	Відгук	Середнє 10/10	Середнє 20/20	$D_ф$	$D_з$	F	Різниця 20/20 і 10/10
480	P_{skip}	0,0589	0,0537	0,022	0,000565	39,0	−0,0052 ± 0,0016
	$\bar{Q}_1$, од.	24,75	28,04	8630	52,6	164,2	3,28 ± 0,50
	$\bar{Q}_2$, од.	17,48	20,72	8410	58,9	142,9	3,24 ± 0,53
	$Q_{1,max}$, од.	38,56	46,08	45 300	129	351,4	7,52 ± 0,79
	$Q_{2,max}$, од.	40,53	45,18	17 300	140	123,3	4,64 ± 0,82
	$П$, виробів/год	56,46	56,78	79,3	2,04	39,0	0,31 ± 0,10
2400	P_{skip}	0,0200	0,0188	0,00126	0,000115	10,9	−0,0013 ± 0,0007
	$\bar{Q}_1$, од.	38,53	41,22	5800	282	20,6	2,69 ± 1,16
	$\bar{Q}_2$, од.	31,50	33,32	2630	228	11,5	1,81 ± 1,05
	$Q_{1,max}$, од.	61,97	69,24	42 300	676	62,6	7,27 ± 1,80
	$Q_{2,max}$, од.	71,84	74,98	7840	617	12,7	3,13 ± 1,72
	$П$, виробів/год	58,80	58,87	4,52	0,414	10,9	0,08 ± 0,04

Для всіх відгуків і обох значень $T_{мод}$ $F > F_{кр}$ (найменше значення 10,9), тобто вплив схеми комплектації статистично значущий. У колонці різниці наведено 95 % довірчий інтервал різниці середніх; жоден інтервал не містить нуля.

Статистична значущість не означає практичної. Кожна пропущена секція базового варіанта залишає неукомплектованими 10 виробів кожного типу, варіанта 20/20 – 20, тому кількість неукомплектованих за зміну виробів дорівнює $P_{skip}\cdot 480$. За $T_{мод} = 480$ хв базовий варіант пропускає в середньому 2,83 секції з 48 за зміну, варіант 20/20 – 1,29 з 24, а кількість неукомплектованих виробів кожного типу становить 28,3 і 25,8. Перехід на 20/20 зберігає 2,5 виробу за зміну, тобто 0,5 % змінного надходження, і підвищує продуктивність на 0,31 ± 0,10 виробу/год. Водночас середні черги зростають на 13–19 %, максимальні – на 11–20 %, а кожна порожня секція означає вдвічі більшу втрату виробів. За $T_{мод}$ = 2400 хв різниця в неукомплектованих виробах зменшується до 0,6 виробу за зміну, а черги у варіанті 20/20 так само більші. Отже, вплив схеми статистично значущий, але виграш у комплектації малий порівняно зі зростанням незавершеного виробництва; висновок щодо доцільності переходу сформульовано в розділі 5.

4.7 Перехідний період і кількість прогонів моделі з накопичувачем

У моделі з накопичувачем черги обмежені місткостями L_1 і L_2, тому сталий режим існує, і для неї виконано повну процедуру визначення перехідного періоду і довжини прогону. Перехідний період визначено для точки з найбільшими місткостями (30, 80), у якій черги встановлюються найповільніше. Виконано 400 прогонів, для кожного обчислено середню за часом довжину черги у вікнах по 50 хв, а потім середнє за прогонами (рис. 4.3). Миттєві значення черги в моменти, кратні 10 хв, для цього непридатні: вони припадають на момент подачі секції, коли черга максимальна перед комплектацією.

Кінцем перехідного періоду вважається момент, після якого обидві криві не виходять за межі ±5 % від сталого рівня (виділені смуги на рисунку). Він становить 2500 хв; із запасом прийнято $T_{\text{п}} = 3000$ хв. Статистика збирається після $T_{\text{п}}$.

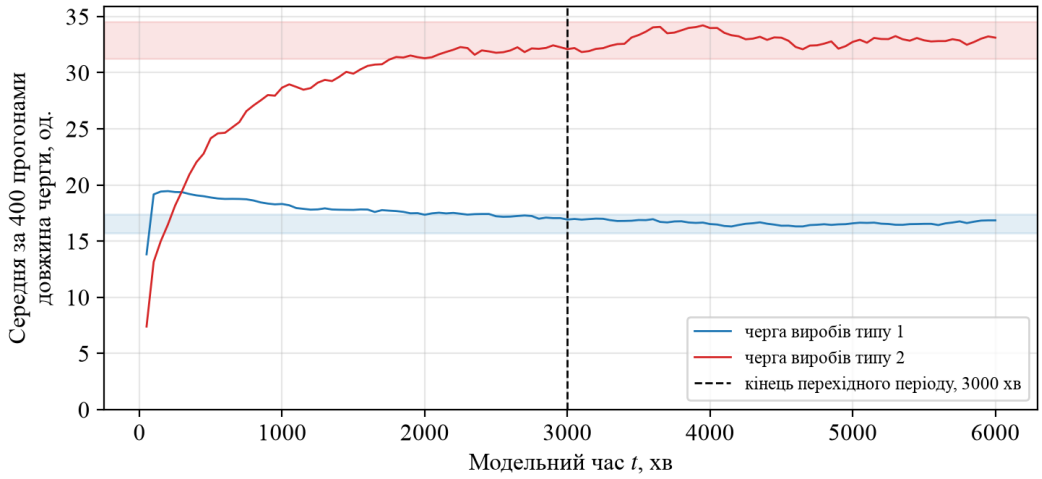


Рисунок 4.3 – Встановлення черг у моделі з накопичувачем (30, 80)

Довжину прогону визначено за умовою (4.8) для точки (20, 60), у околі якої ймовірність пропуску досягає рівня 0,02, за 5 прогонами тривалістю 100 000 хв (рис. 4.4). Для окремого прогону прийнято ε = 0,005 для P_{skip} і ε = 2 вироби для середніх черг.

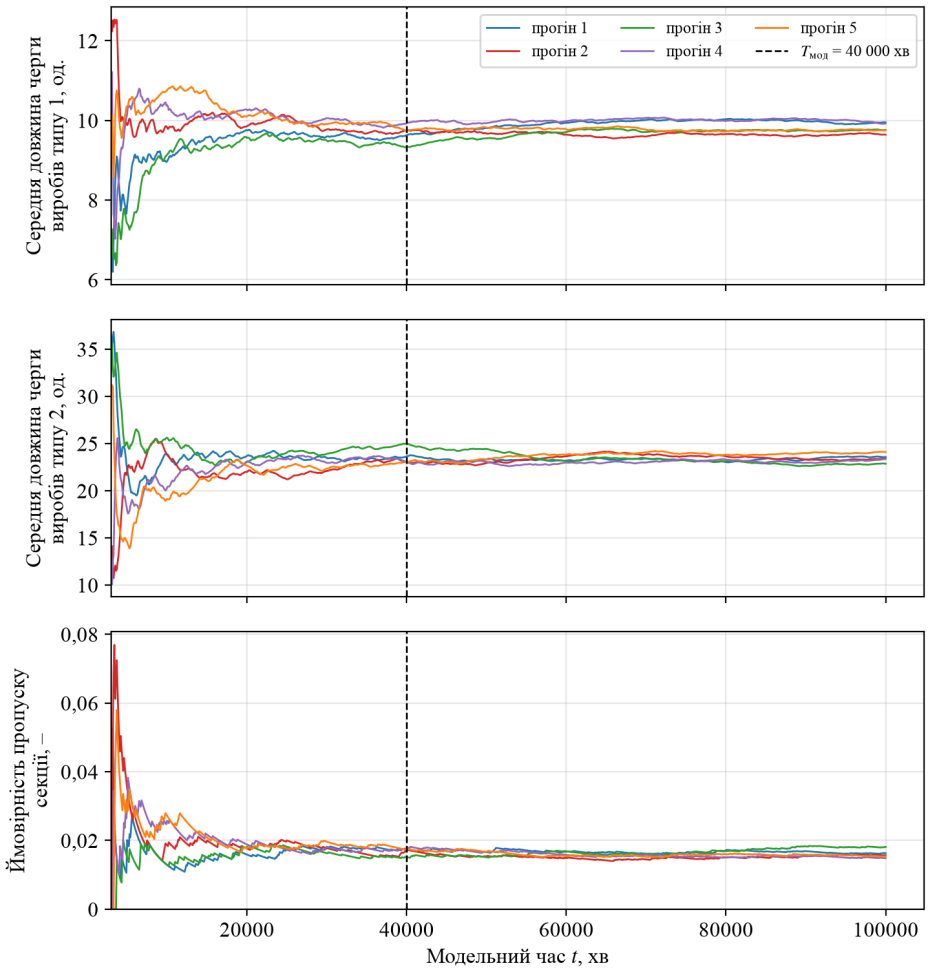


Рисунок 4.4 – Поточні середні значення відгуків, накопичувач (20, 60)

Умова виконується для P_{skip} з 18 400 хв, для $\bar{Q}_1$ – з 6900 хв, для $\bar{Q}_2$ – з 35 700 хв, тому прийнято $T_{мод}$ = 40 000 хв. Для точки (30, 80) розкид $\bar{Q}_2$ за 100 000 хв ще становить 3,2 виробу, тобто умова не виконується в межах спостереження; потрібну точність середнього там забезпечує кількість прогонів.

Кількість прогонів визначено за формулою (4.9) для точності ε = 0,002 для P_{skip} і ε = 1 виріб для середніх черг за 20 пробними прогонами тривалістю $T_{мод}$ = 40 000 хв (табл. 4.7).

Таблиця 4.7 – Кількість прогонів моделі з накопичувачем

(L_1, L_2), од.	Відгук	ε	σ	p
(20, 60)	P_{skip}	0,002	0,0016	20
	$\bar{Q}_1$, од.	1	0,21	20
	$\bar{Q}_2$, од.	1	0,78	20
(30, 80)	P_{skip}	0,002	0,0018	20
	$\bar{Q}_1$, од.	1	0,60	20
	$\bar{Q}_2$, од.	1	2,44	119

Найбільше значення 119 отримано для $\bar{Q}_2$ у точці (30, 80), тому в кожній точці серії виконано p = 120 прогонів.

4.8 Вплив місткості накопичувача на ймовірність пропуску секції

Серію експериментів проведено для всіх 24 комбінацій місткостей L_1 = 15, 20, 25, 30 і L_2 = 30, 40, 50, 60, 70, 80 виробів, $T_{мод}$ = 40 000 хв, $T_п$ = 3000 хв, 120 прогонів у кожній точці. Залежність ймовірності пропуску від місткостей показано на рисунку 4.5. Повні результати перебору наведено в додатку Е: середні значення ймовірності пропуску – у таблиці Е.1, середні довжини черг – на рисунку Е.1. Півширина 95 % довірчого інтервалу P_{skip} в усіх точках не перевищує 0,0009.

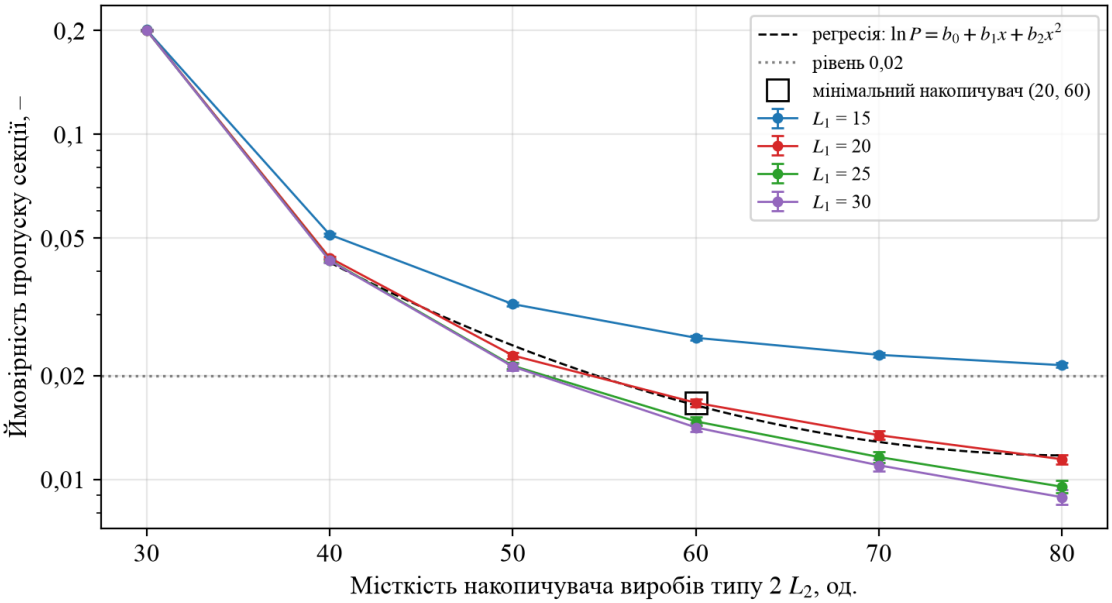


Рисунок 4.5 – Ймовірність пропуску секції залежно від місткостей

Ймовірність пропуску визначається передусім місткістю L_2. За L_1 = 15 навіть L_2 = 80 дає P_{skip} = 0,0214, а для $L_1 \geq 20$ збільшення L_1 мало змінює P_{skip} (за L_2 = 60 – 0,0167, 0,0147, 0,0141): додаткові місця для виробів типу 1 переважно заповнені (рис. Е.1), а пропуски спричиняють рідкі великі партії типу 2. Місткість L_2 = 30 дорівнює партії типу 2 плюс одній секції, тому нову партію можна почати лише тоді, коли в накопичувачі залишилося не більше 10 виробів; надходження типу 2 стає повільнішим за комплектацію і $P_{skip} \approx$ 0,20 незалежно від L_1 – це окремий режим.

Прийнято допустимий рівень $P_{skip} \le 0{,}02$, що відповідає приблизно одній порожній секції за зміну з 48. Повний перебір серії показує, що найменшу загальну місткість $L_1 + L_2$, за якої верхня межа довірчого інтервалу P_{skip} не перевищує цього рівня, має точка (20, 60): $P_{skip} = 0{,}0167 \pm 0{,}0004$.

Для кількісного опису залежності P_{skip} від L_2 виконано регресійний аналіз за $L_1 = 20$ на інтервалі $L_2 \in [40; 80]$ (5 точок); точку $L_2 = 30$ виключено як окремий режим. Фактор нормалізовано до інтервалу $[-1; 1]$ [3]:

$$x = \frac{L_2 - X_0}{\Delta}, \quad X_0 = 60, \quad \Delta = 20 \tag{4.13}$$

Розглянуто чотири гіпотези про вид залежності, лінійні за параметрами: лінійну і квадратичну залежність P_{skip} і $\ln P_{skip}$ від x. Параметри визначено методом найменших квадратів:

$$b = (X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}Y \tag{4.14}$$

де b – вектор параметрів рівняння регресії;

X – матриця, рядки якої містять 1, x, x^2 для точок серії;

Y – вектор значень P_{skip} (або $\ln P_{skip}$) у точках серії.

Якість рівнянь оцінено індексом кореляції, обчисленим для значень P_{skip}:

$$R = \sqrt{1 - \frac{\sum_{j=1}^{n}\left(P_j - \hat{P}_j\right)^2}{\sum_{j=1}^{n}\left(P_j - \overline{P}\right)^2}} \tag{4.15}$$

де P_j – ймовірність пропуску в j-й точці за моделлю;

$\hat{P}_j$ – значення за рівнянням регресії;

n – кількість точок.

Параметри і показники якості рівнянь наведено в таблиці 4.8.

Таблиця 4.8 – Рівняння регресії ймовірності пропуску секції

Рівняння	b_0	b_1	b_2	R	Найбільша похибка P
$P = b_0 + b_1x$	0,02165	−0,01486	–	0,8933	0,0074
$P = b_0 + b_1x + b_2x^2$	0,01579	−0,01486	0,01171	0,9856	0,0033
$\ln P = b_0 + b_1x$	−3,956	−0,6443	–	0,9436	0,0074
$\ln P = b_0 + b_1x + b_2x^2$	−4,111	−0,6443	0,3093	0,9966	0,0016

Усі рівняння мають $R > 0{,}7$, але лише квадратична залежність логарифма описує дані з похибкою, меншою за різницю між сусідніми точками серії: $R = 0{,}9966$, найбільша похибка 0,0016. Рівняння регресії має вигляд

$$\ln P = -4{,}111 - 0{,}644x + 0{,}309x^2 \qquad (4.16)$$

Рівняння побудовано за 5 точками з трьома параметрами, тому воно придатне лише для інтерполяції в межах досліджуваного інтервалу. За рівнянням (4.16) рівень 0,02 досягається за $L_2 \approx 54{,}5$. Довжина черги типу 2 завжди кратна 10 (партії по 20 і секції по 10 виробів), тому місткість між двома кратними 10 значеннями працює як менше з них, і значення треба округлити вгору до $L_2 = 60$. Це збігається з результатом повного перебору: найменша достатня місткість (20, 60). Варіанти накопичувача порівнюються з базовою системою в розділі 5. Код експериментального дослідження наведено в додатках В і Г.

4.9 Висновок до розділу

Теоретичний розрахунок показав, що коефіцієнт завантаження системи дорівнює 1, а різниця черг двох типів не залежить від конвеєра і зростає пропорційно кореню з часу, тому черги базової системи і варіанта 20/20 не мають сталого режиму, що підтверджено дослідженням перехідного періоду. Для цих моделей застосовано моделювання на скінченному інтервалі $T_{\text{мод}} = 480$ хв (додатково 2400 хв) з 1600 прогонами, кількість яких обґрунтовано за формулою (4.9). За зміну ймовірність пропуску секції становить $0{,}0589 \pm 0{,}0012$, середні черги – 24,8 і 17,5 виробу, середні максимальні – 38,6 і 40,5 виробу. Дисперсійний аналіз виявив статистично значущий вплив схеми комплектації на всі відгуки, але практичний виграш варіанта 20/20 у комплектації малий, а черги в ньому більші. Для моделі з накопичувачем визначено перехідний період 3000 хв, довжину прогону 40 000 хв і 120 прогонів у точці; повний перебір і регресійний аналіз дали найменший накопичувач (20, 60) для рівня $P_{skip} \le 0{,}02$.

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ

5.1 Характеристики роботи конвеєра

Обидва постачальники в середньому привозять рівно стільки виробів, скільки конвеєр здатен укомплектувати: по одному виробу кожного типу за хвилину. Резерву потужності немає, тому будь-яка випадкова затримка партії одразу перетворюється на порожню секцію, а вироби, що надійшли раніше за потрібне, накопичуються в черзі.

Статистичні оцінки характеристик базової системи наведено в таблиці 4.5. Для цеху вони означають, що за зміну ($T_{\text{мод}}$ = 480 хв) приблизно 3 секції з 48 проходять порожніми, а перед конвеєром постійно очікують кілька десятків виробів кожного типу. Порожні секції виникають переважно на початку зміни, коли запасу виробів ще немає і конвеєр чекає на першу партію другого типу, тому за тиждень безперервної роботи частка пропусків менша, а черги, навпаки, більші: різниця черг двох типів конвеєром не зменшується і зростає приблизно як $0{,}5\cdot\sqrt{T}$ виробу (підрозділ 4.2). Сталого рівня черг у цій системі немає, і цього не змінює жодна швидкість конвеєра: швидший конвеєр лише частіше проходить порожнім, а повільніший збільшує обидві черги.

5.2 Доцільність переходу на комплектацію по 20 виробів

За дисперсійним аналізом (підрозділ 4.6, табл. 4.6) перехід на секції по 20 виробів кожні 20 хв впливає на всі характеристики, і цей вплив не пояснюється випадковим розкидом. Проте за зміну він зберігає лише близько 0,5 % змінного надходження виробів, а за тиждень ще менше. Водночас варіант 20/20 погіршує інші характеристики:

– середні й найбільші черги зростають, тобто біля конвеєра потрібно більше місця і більше виробів перебуває в незавершеному виробництві;

– кожна порожня секція означає втрату комплекту з 20 виробів кожного типу замість 10, тому робоче місце, що чекає на секцію, простоює вдвічі довше;

– секції подаються вдвічі рідше і вдвічі більшими, що вимагає переобладнання конвеєра і робочих місць під більший комплект.

Отже, перехід на комплектацію по 20 виробів кожного типу недоцільний: малий виграш у комплектації не виправдовує більших запасів перед конвеєром і змін в обладнанні, а основний недолік системи – необмежене зростання черг – цей варіант не усуває.

5.3 Обмежений накопичувач з блокуванням постачання

Оскільки черги не можна обмежити швидкістю конвеєра, запропоновано обмежити саме запас виробів. Перед конвеєром для кожного типу встановлюється накопичувач на L_1 і L_2 місць, а постачальник починає виготовлення нової партії лише тоді, коли в накопичувачі є місце для всієї партії; інакше він чекає (принцип «канбан»). Мережу Петрі такої системи наведено в підрозділі 2.5, вибір місткостей – у підрозділі 4.8. Розглянуто два варіанти:

– накопичувач (20, 60) – найменша загальна місткість, за якої ймовірність пропуску в сталому режимі не перевищує 0,02;

– накопичувач (25, 80) – на 25 місць більший, з меншою втратою комплектації.

Пробні прогони обох варіантів і базової системи виконано з однаковими послідовностями випадкових чисел: 1600 прогонів за $T_{\text{мод}}$ = 480 і 2400 хв з порожнім стартом і 120 прогонів у сталому режимі ($T_{\text{мод}}$ = 40 000 хв, $T_{\text{п}}$ = 3000 хв). Результати наведено в таблиці 5.1 і на рисунку 5.1 (для різниці продуктивності вказано 95 % довірчий інтервал; простій – частка часу, протягом якого постачальник типу 1 / типу 2 чекає на місце в накопичувачі).

Таблиця 5.1 – Порівняння накопичувачів з базовою системою

$T_{\text{мод}}$, хв	Модель	P_{skip}	$\bar{Q}_1$, од.	$\bar{Q}_2$, од.	$Q_{1,max}$, од.	$Q_{2,max}$, од.	$П$, виробів/год	Різниця $П$, виробів/год	Простій, %
480	базова	0,0589	24,8	17,5	38,6	40,5	56,46	–	–
	(20, 60)	0,0597	10,9	17,0	20,0	39,5	56,42	−0,04 ± 0,08	4,4 / 0,6
	(25, 80)	0,0602	15,1	17,4	24,6	40,7	56,39	−0,08 ± 0,08	3,5 / 0,0
2400	базова	0,0200	38,5	31,5	62,0	71,8	58,80	–	–
	(20, 60)	0,0253	10,1	21,8	20,0	49,9	58,48	−0,31 ± 0,03	2,2 / 1,4
	(25, 80)	0,0218	13,8	27,8	25,0	62,6	58,69	−0,10 ± 0,03	1,7 / 0,6

$T_{мод}$, хв	Модель	P_{skip}	$\bar{Q}_1$, од.	$\bar{Q}_2$, од.	$Q_{1,max}$, од.	$Q_{2,max}$, од.	$П$, виробів/год	Різниця $П$, виробів/год	Простій, %
40 000, сталий режим	базова	0,0028	108,6	112,2	183,5	249,5	59,83	–	–
	(20, 60)	0,0167	9,8	23,1	20,0	56,2	59,00	−0,83 ± 0,03	1,6 / 1,7
	(25, 80)	0,0095	12,8	33,7	25,0	72,9	59,43	−0,40 ± 0,03	0,9 / 1,0

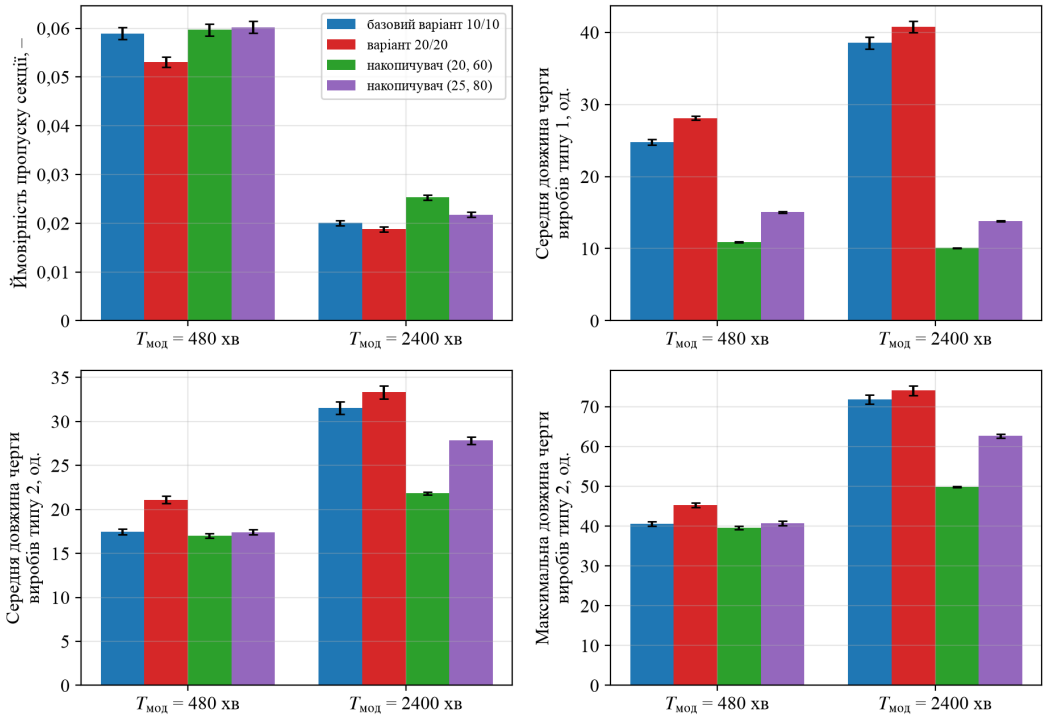


Рисунок 5.1 – Порівняння варіантів системи, 1600 прогонів

У межах однієї зміни накопичувач (20, 60) практично не змінює ймовірність пропуску (різниця 0,0007 ± 0,0013) і продуктивність (різниця −0,04 ± 0,08 виробу/год): обидва інтервали містять нуль, тобто різниця менша за випадковий розкид. Черга виробів типу 1 помітно зменшується, а черга типу 2 за зміну майже не змінюється, бо рідко встигає дійти до 60 виробів. Постачальник типу 1 чекає 4,4 % часу: на початку зміни вироби типу 1 заповнюють накопичувач, поки не прийде перша партія типу 2.

За тиждень ($T_{мод}$ = 2400 хв) накопичувач (20, 60) уже гірший за базову систему: ймовірність пропуску більша на 0,0052 ± 0,0006 (приблизно на 0,25 порожньої секції за зміну), продуктивність менша на 0,31 виробу/год (0,5 %), натомість черги обох типів помітно менші.

Головна відмінність проявляється за тривалої роботи. У базовій системі черги зростають без обмеження (приблизно як $0{,}5 \cdot \sqrt{T}$), а з накопичувачем вони

ніколи не перевищують його місткості і мають сталі середні значення. Ціна цього – втрата частини комплектації: у сталому режимі 0,8 і 0,5 порожньої секції за зміну для (20, 60) і (25, 80), а продуктивність менша на 1,4 % і 0,7 % відповідно. Вироби, яких постачальник не виготовив під час очікування, у базовій системі лише поповнили б чергу, тому справжньою втратою є саме зменшення продуктивності.

Накопичувач (25, 80) займає на 25 місць більше, але втрачає вдвічі менше: за зміну різниця з базовою системою на межі випадкового розкиду (0,0013 ± 0,0013 для ймовірності пропуску), за тиждень продуктивність менша лише на 0,2 %, у сталому режимі – на 0,7 %.

Отже, накопичувач з блокуванням постачання доцільний, якщо для цеху важливо мати передбачуваний і обмежений запас виробів перед конвеєром. Варіант (20, 60) рекомендується, коли дорожчим є складське місце, варіант (25, 80) – коли дорожчою є кожна неукомплектована секція. Для роботи короткими змінами з порожнім стартом обидва варіанти зберігають продуктивність базової системи і зменшують запас виробів типу 1.

5.4 Напрями подальших досліджень

Результати роботи можна уточнити й розширити в таких напрямах:

1) моделювання реального графіка роботи цеху: кількох змін поспіль з перенесенням залишків черг, перерв і зупинок постачальників;

2) економічна модель вибору місткостей: якщо задати вартість місця в накопичувачі, порожньої секції і простою постачальника, місткості L_1 і L_2 можна визначити як оптимальні за сумарними витратами, а не за заданим рівнем ймовірності пропуску;

3) зменшення партії виробів типу 2 (наприклад, 10 виробів кожні 10 хв): різниця черг двох типів зростає тим повільніше, чим менші і регулярніші партії, тому такий захід може зменшити і пропуски, і потрібну місткість накопичувача;

4) використання фактичних розподілів інтервалів надходження зі статистики цеху замість рівномірних.

ВИСНОВКИ

У роботі побудовано імітаційну модель комплектувального конвеєра автоматизованого складального цеху. Систему описано концептуальною моделлю і формалізовано стохастичною мережею Петрі з восьми позицій і п’яти переходів, у якій пакетне надходження виробів відображено кратними дугами, а вибір між комплектацією і пропуском секції – пріоритетами переходів. Розроблено власний алгоритм імітації стохастичної мережі Петрі з просуванням часу за найближчою подією і реалізовано його мовою Python. Коректність алгоритму перевірено на 7 наборах вхідних даних, для яких очікувані значення обчислено аналітично, і автоматизованими тестами.

Отримано такі результати:

1) за одну зміну ($T_{\text{мод}}$ = 480 хв, 1600 прогонів) ймовірність пропуску секції становить 0,0589 ± 0,0012, середні черги виробів типу 1 і типу 2 – 24,8 і 17,5 виробу, середні максимальні – 38,6 і 40,5 виробу; за тиждень ($T_{\text{мод}}$ = 2400 хв) ймовірність пропуску зменшується до 0,0200, а черги зростають;

2) середня інтенсивність надходження виробів кожного типу дорівнює максимальній швидкості комплектації, а різниця черг двох типів не залежить від конвеєра і зростає приблизно як $0{,}5\cdot\sqrt{T}$, тому черги базової системи не мають сталого режиму за будь-якої швидкості конвеєра; це встановлено теоретично і підтверджено моделюванням;

3) перехід на комплектацію по 20 виробів зменшує ймовірність пропуску з 0,0589 до 0,0537, але збільшує середні черги на 13–19 %; вплив статистично значущий, проте практичний виграш становить близько 0,5 % виробів за зміну, тому перехід визнано недоцільним;

4) запропоновано обмежений накопичувач з блокуванням постачання; найменший накопичувач (20, 60) для рівня ймовірності пропуску 0,02 визначено повним перебором і регресійним аналізом; у сталому режимі ($T_{\text{мод}}$ = 40 000 хв, 120 прогонів) черги не перевищують місткостей накопичувачів, а продуктивність зменшується на 1,4 % для (20, 60) і на 0,7 % для (25, 80), тоді як у межах зміни різниця з базовою системою незначуща.

Модель ґрунтується на таких основних припущеннях і спрощеннях: інтервали між партіями незалежні й рівномірно розподілені, партія надходить цілком в один момент, комплектація миттєва, неповних комплектів не буває, зміна починається з порожніх накопичувачів, відмови обладнання і перерви не розглядаються. Ці припущення відповідають умові завдання і не впливають на виявлені закономірності, але кількісні результати для реального цеху потребують перевірки на фактичних даних.

Моделювання можна вважати успішним: модель відповідає на всі питання завдання, результати узгоджуються з теоретичними оцінками, а точність кожної оцінки обґрунтовано кількістю прогонів. Водночас з’ясувалося, що поставлена в завданні задача визначення середніх і максимальних черг не має однозначної відповіді без зазначення тривалості роботи, тому всі результати наведено разом з $T_{\text{мод}}$.

Перевага імітаційного підходу полягає в тому, що він не потребує спрощень, необхідних для аналітичних моделей: пакетні надходження, рівномірні інтервали, детермінований конвеєр і умова одночасної наявності двох ресурсів відтворюються безпосередньо. Модель дає змогу оцінити не лише середні значення, а й розкид, максимальні значення і поведінку системи в часі, а також швидко перевірити модифікації. Розроблений алгоритм універсальний: він імітує будь-яку стохастичну мережу Петрі з затримками, пріоритетами, ймовірностями і багатоканальними переходами, тому модель з накопичувачем отримано додаванням двох позицій і відповідних дуг без зміни алгоритму.

Розробка моделі на основі стохастичної мережі Петрі виявилася нескладною: опис системи займає 13 рядків коду – по одному на кожну позицію і перехід, а умова комплектації природно виражається кратними дугами. Основні труднощі пов’язані з алгоритмом імітації – правильним порядком подій в один момент часу, обробкою переходів з нульовою затримкою і збиранням статистики, зваженої за часом. Ці питання розв’язано один раз в алгоритмі, і для конкретної моделі вони вже не виникають. Найбільших зусиль потребувало не програмування, а планування експериментів для системи без сталого режиму.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Стеценко І. В. Моделювання систем : навч. посіб. Черкаси : ЧДТУ, 2010. 399 с.

2. Томашевський В. М. Моделювання систем. Київ : Видавнича група BHV, 2005. 352 с.

3. Стеценко І. В., Дифучина О. Ю., Дифучин А. Ю. Моделювання систем. Курсова робота : рекомендації до виконання курсової роботи : навч. посіб. для здобувачів ступеня бакалавра за освітньою програмою «Інженерія програмного забезпечення інформаційних систем» спец. 121 Інженерія програмного забезпечення. Київ : КПІ ім. Ігоря Сікорського, 2024. 109 с. URL: https://ela.kpi.ua/handle/123456789/69544 (дата звернення: 08.11.2026).

4. Python 3.12 Documentation / Python Software Foundation. URL: https://docs.python.org/3.12/ (дата звернення: 08.11.2026).

5. NumPy Documentation : NumPy v2.5 Manual. URL: https://numpy.org/doc/2.5/ (дата звернення: 08.11.2026).

6. SciPy Documentation : SciPy v1.18.0 Manual. URL: https://docs.scipy.org/doc/scipy-1.18.0/ (дата звернення: 08.11.2026).

7. Matplotlib Documentation : Matplotlib 3.11.2. URL: https://matplotlib.org/3.11.2/ (дата звернення: 08.11.2026).

8. Ross S. M. Introduction to Probability Models. 13th ed. London : Academic Press, 2024. 870 p.

ДОДАТОК А. Код алгоритму імітації

Наведено пакет src/petri (без класів Exp і Norm, які модель не використовує) і модель src/model/conveyor.py; повний код – у репозиторії https://github.com/kqjqxb/Computer-simulation-course-work.

Файл src/petri/rng.py

```
"""Random delays in the guide's notation.

const(d)    - deterministic value d
unif(d, s)  - uniform on (d - s, d + s)
"""
import numpy as np

def make_rng(seed=None):
    return np.random.default_rng(seed)

class Delay:
    def sample(self, rng):
        raise NotImplementedError

    def __call__(self, rng):
        return self.sample(rng)

class Const(Delay):
    def __init__(self, d):
        self.d = float(d)

    def sample(self, rng):
        return self.d

    def __repr__(self):
        return f"{self.d:g}"

class Unif(Delay):
    def __init__(self, d, s):
        if s < 0 or d - s < 0:
            raise ValueError("unif(d, s) needs 0 <= s <= d")
        self.d, self.s = float(d), float(s)

    def sample(self, rng):
        # unif(d, s) = d - s + 2s * r,  r ~ U(0, 1)
        return self.d - self.s + 2.0 * self.s * rng.random()

    def __repr__(self):
        return f"unif({self.d:g}, {self.s:g})"

def const(d):
```

```
    return Const(d)

def unif(d, s):
    return Unif(d, s)

def as_delay(value):
    """Accept a Delay, a callable rng -> float, or a number."""
    if isinstance(value, Delay) or callable(value):
        return value
    return Const(value)
```

Файл src/petri/place.py

```
from .stats import TimeWeightedStat

class Place:
    """Petri net place: a marker count plus time-weighted
    statistics."""

    def __init__(self, name, markers=0):
        if markers < 0:
            raise ValueError("marker count must be non-negative")
        self.name = name
        self.initial = int(markers)
        self.markers = int(markers)
        self.stat = TimeWeightedStat()

    def reset(self):
        self.markers = self.initial
        self.stat = TimeWeightedStat()

    def __repr__(self):
        return f"Place({self.name!r}, {self.markers})"
```

Файл src/petri/arc.py

```
from dataclasses import dataclass

from .place import Place

@dataclass(frozen=True)
class Arc:
    """Arc between a place and a transition; weight = arc
    multiplicity k."""

    place: Place
    weight: int = 1

    def __post_init__(self):
        if self.weight < 1:
            raise ValueError("arc weight must be >= 1")
```

Файл src/petri/transition.py

```
from bisect import insort

from .rng import as_delay

class Transition:
    """Timed transition with a list of exit moments
    (multichannel).

    priority: larger value fires first when transitions conflict.
    probability: relative weight among enabled transitions of
    equal priority.
    """

    def __init__(self, name, delay=0.0, priority=0,
    probability=1.0):
        if probability <= 0:
            raise ValueError("probability must be positive")
        self.name = name
        self.delay = as_delay(delay)
        self.priority = int(priority)
        self.probability = float(probability)
        self.inputs = []
        self.outputs = []
        self.exits = []  # sorted exit moments of busy channels

    def is_enabled(self):
        """X(M, T): every input place holds at least the arc
    weight."""
        return all(arc.place.markers >= arc.weight for arc in
    self.inputs)

    def fire_in(self, t, rng):
        """Take markers from input places and schedule an exit."""
        for arc in self.inputs:
            arc.place.markers -= arc.weight
        exit_time = t + float(self.delay(rng))
        insort(self.exits, exit_time)
        return exit_time

    def fire_out(self, t):
        """Release every channel whose exit moment is t. Returns
    how many."""
        n = 0
        while self.exits and self.exits[0] <= t:
            self.exits.pop(0)
            n += 1
        for arc in self.outputs:
            arc.place.markers += arc.weight * n
        return n
```

```
    @property
    def next_exit(self):
        return self.exits[0] if self.exits else float("inf")

    @property
    def busy(self):
        return len(self.exits)

    def reset(self):
        self.exits = []

    def __repr__(self):
        return f"Transition({self.name!r}, delay={self.delay!r},
    r={self.priority}, b={self.probability:g})"
```

Файл src/petri/net.py

```
from .arc import Arc
from .place import Place
from .transition import Transition

class PetriNet:
    def __init__(self, name=""):
        self.name = name
        self.places = {}
        self.transitions = {}
        self.params = {}

    def add_place(self, name, markers=0):
        if name in self.places:
            raise ValueError(f"place {name!r} already exists")
        place = Place(name, markers)
        self.places[name] = place
        return place

    def add_transition(self, name, delay=0.0, inputs=None,
    outputs=None,
                       priority=0, probability=1.0):
        """inputs / outputs: {place_name: arc_weight}."""
        if name in self.transitions:
            raise ValueError(f"transition {name!r} already
    exists")
        tr = Transition(name, delay, priority, probability)
        for pname, w in (inputs or {}).items():
            tr.inputs.append(Arc(self.places[pname], w))
        for pname, w in (outputs or {}).items():
            tr.outputs.append(Arc(self.places[pname], w))
        self.transitions[name] = tr
        return tr

    def marking(self):
        return {name: p.markers for name, p in
    self.places.items()}
```

```
    def enabled(self):
        return [tr for tr in self.transitions.values() if
    tr.is_enabled()]

    def next_event_time(self):
        return min((tr.next_exit for tr in
    self.transitions.values()), default=float("inf"))

    def reset(self):
        for p in self.places.values():
            p.reset()
        for tr in self.transitions.values():
            tr.reset()
```

Файл src/petri/stats.py

```
import math

class TimeWeightedStat:
    """Mean, std, min and max of a piecewise-constant value over
    time.

    mean = sum(y_s * dt_s) / sum(dt_s)
    var  = sum(y_s^2 * dt_s) / sum(dt_s) - mean^2
    min / max are taken over values held for a positive time.
    """

    def __init__(self):
        self.total_time = 0.0
        self.area = 0.0
        self.area_sq = 0.0
        self.max = None
        self.min = None

    def add(self, value, dt):
        if dt <= 0:
            return
        self.total_time += dt
        self.area += value * dt
        self.area_sq += value * value * dt
        self.max = value if self.max is None else max(self.max,
    value)
        self.min = value if self.min is None else min(self.min,
    value)

    @property
    def mean(self):
        return self.area / self.total_time if self.total_time > 0
    else math.nan

    @property
```

```
    def std(self):
        if self.total_time <= 0:
            return math.nan
        var = self.area_sq / self.total_time - self.mean ** 2
        return math.sqrt(max(var, 0.0))
```

Файл src/petri/simulator.py

```
"""Next-event simulation of a stochastic Petri net.

One step:
  1. t_next = min exit moment over all transitions;
  2. time-weighted statistics are collected on [t, t_next);
  3. t := t_next;
  4. out-phase: transitions whose exit moment is t release markers
     (transitions in random order);
  5. in-phase: while some transition is enabled, fire one of the
     enabled transitions with the highest priority (ties broken by
     the probability weights);
  6. zero-delay transitions fired in step 5 exit at the same t, so
     steps 4-5 repeat until no exit is left at t.
"""
from collections import Counter

from .rng import make_rng

MAX_FIRINGS_PER_INSTANT = 10_000

class Simulator:
    def __init__(self, net, seed=None, protocol=None,
    observer=None):
        self.net = net
        self.rng = make_rng(seed)
        self.protocol = protocol
        self.observer = observer  # called as observer(sim) after
    each instant
        self.t = 0.0
        self.warmup = 0.0
        self.fired_in = Counter()   # all firings
        self.fired_out = Counter()  # all exits
        self.window_out = Counter()  # exits in (warmup, t_end]

    # --- state change ----------------------------------------
    --------
    def _out_phase(self):
        ready = [tr for tr in self.net.transitions.values() if
    tr.next_exit <= self.t]
        self.rng.shuffle(ready)
        for tr in ready:
            n = tr.fire_out(self.t)
            self.fired_out[tr.name] += n
            if self.t > self.warmup or self.warmup == 0:
                self.window_out[tr.name] += n
```

```
        if self.protocol is not None:
            for _ in range(n):
                self.protocol.record(self.t, "OUT", tr,
self.net)
    return bool(ready)

def _choose(self, enabled):
    top = max(tr.priority for tr in enabled)
    group = [tr for tr in enabled if tr.priority == top]
    if len(group) == 1:
        return group[0]
    weights = [tr.probability for tr in group]
    total = sum(weights)
    idx = self.rng.choice(len(group), p=[w / total for w in
weights])
    return group[idx]

def _in_phase(self):
    """Fire enabled transitions one by one; returns how many
fired."""
    fired = 0
    while fired <= MAX_FIRINGS_PER_INSTANT:
        enabled = self.net.enabled()
        if not enabled:
            return fired
        tr = self._choose(enabled)
        tr.fire_in(self.t, self.rng)
        self.fired_in[tr.name] += 1
        if self.protocol is not None:
            self.protocol.record(self.t, "IN", tr, self.net)
        fired += 1
    return fired

def _settle(self):
    """Process the current instant until nothing more happens
at t."""
    fired = self._in_phase()
    while self.net.next_event_time() <= self.t:
        if fired > MAX_FIRINGS_PER_INSTANT:
            raise RuntimeError(f"net does not settle at
t={self.t}")
        self._out_phase()
        fired += self._in_phase()

# --- statistics ----------------------------------------------
--------
def _collect(self, t_from, t_to):
    lo = max(t_from, self.warmup)
    dt = t_to - lo
    if dt <= 0:
        return
    for place in self.net.places.values():
```

```
        place.stat.add(place.markers, dt)

# --- main loop ---------------------------------------------------
--------
def run(self, t_end, warmup=0.0, sample_every=None,
on_sample=None):
    """Simulate on [0, t_end]; statistics are collected on
[warmup, t_end].

    If sample_every is given, on_sample(sim) is called at t =
sample_every,
    2 * sample_every, ... <= t_end with the statistics
collected up to
    that moment (before the events of that moment are
processed). Used
    for running means in the transient-period study.
    """
    if not 0 <= warmup < t_end:
        raise ValueError("need 0 <= warmup < t_end")
    self.warmup = float(warmup)
    self.net.reset()
    self.t = 0.0
    self.fired_in.clear()
    self.fired_out.clear()
    self.window_out.clear()
    next_sample = sample_every if sample_every else
float("inf")

    self._settle()
    if self.observer:
        self.observer(self)
    while True:
        t_next = self.net.next_event_time()
        bound = min(t_next, t_end)
        while next_sample <= bound:
            self._collect(self.t, next_sample)
            self.t = next_sample
            on_sample(self)
            next_sample += sample_every
        if t_next > t_end:
            self._collect(self.t, t_end)
            self.t = float(t_end)
            break
        self._collect(self.t, t_next)
        self.t = t_next
        self._out_phase()
        self._settle()
        if self.observer:
            self.observer(self)
    return self
```

```
    # --- results ------------------------------------------------
    --------
    def place_stats(self, name):
        st = self.net.places[name].stat
        return {"mean": st.mean, "std": st.std, "min": st.min,
    "max": st.max}
```

Файл src/model/conveyor.py

```
"""Completing conveyor (variant D2) as a stochastic Petri net.

Base scheme: k = 10 items of each type per section, a section
    every 10 min.
Variant B:   k = 20 items of each type per section, a section
    every 20 min.

Places: G1, G2 - suppliers ready to start a batch; Q1, Q2 - items
    waiting;
C - conveyor drive; S - an empty section at the station; N_fill,
    N_skip -
counters of filled and skipped sections; F1, F2 - free buffer
    places.

Buffer (kanban) modification, buffer=L or (L1, L2): place F_i
    holds the free
places of the type-i buffer. «Надходження i» takes b_i of them
    when it starts
(a supplier begins a batch only if there is room for it, otherwise
    it waits),
«Комплектація» returns k places of each type. Hence Q_i <= L_i at
    any time.
buffer=None gives the base model without the F places.
"""
from src.petri import PetriNet, const, unif

ARRIVAL_1 = "Надходження 1"
ARRIVAL_2 = "Надходження 2"
CONVEYOR = "Рух конвеєра"
FILL = "Комплектація"
SKIP = "Пропуск секції"

def build_conveyor_net(k=10, period=10.0,
                       batch1=5, interval1=unif(5, 1),
                       batch2=20, interval2=unif(20, 7),
                       q1_0=0, q2_0=0, buffer=None):
    limits = _buffer_limits(buffer, k, batch1, batch2, q1_0, q2_0)
    net = PetriNet("conveyor")
    net.params = {"k": k, "period": period, "batch1": batch1,
    "interval1": interval1,
                  "batch2": batch2, "interval2": interval2,
    "buffer": limits}
    net.add_place("G1", 1)
    net.add_place("G2", 1)
```

```
    net.add_place("Q1", q1_0)
    net.add_place("Q2", q2_0)
    net.add_place("C", 1)
    net.add_place("S", 0)
    net.add_place("N_fill", 0)
    net.add_place("N_skip", 0)

    arr1_in, arr2_in = {"G1": 1}, {"G2": 1}
    fill_out = {"N_fill": 1}
    if limits is not None:
        net.add_place("F1", limits[0] - q1_0)
        net.add_place("F2", limits[1] - q2_0)
        arr1_in["F1"] = batch1
        arr2_in["F2"] = batch2
        fill_out.update({"F1": k, "F2": k})

    # generators and the conveyor drive are single-channel: the
    marker in
    # G1 / G2 / C returns only when the transition exits
    net.add_transition(ARRIVAL_1, interval1, arr1_in, {"G1": 1,
    "Q1": batch1})
    net.add_transition(ARRIVAL_2, interval2, arr2_in, {"G2": 1,
    "Q2": batch2})
    net.add_transition(CONVEYOR, const(period), {"C": 1}, {"C": 1,
    "S": 1})

    # FILL and SKIP compete for the section; FILL has the higher
    priority,
    # so a section is skipped only when FILL is not enabled
    net.add_transition(FILL, const(0), {"S": 1, "Q1": k, "Q2": k},
                       fill_out, priority=1)
    net.add_transition(SKIP, const(0), {"S": 1}, {"N_skip": 1},
    priority=0)
    return net

def _buffer_limits(buffer, k, batch1, batch2, q1_0, q2_0):
    if buffer is None:
        return None
    l1, l2 = (buffer, buffer) if isinstance(buffer, int) else
    buffer
    for name, lim, b, q0 in (("L1", l1, batch1, q1_0), ("L2", l2,
    batch2, q2_0)):
        # a smaller buffer can never hold a batch or a full
    section -> deadlock
        if lim < max(b, k):
            raise ValueError(f"{name} = {lim} must be >=
    max(batch, k) = {max(b, k)}")
        if q0 > lim:
            raise ValueError(f"initial queue {q0} exceeds {name} =
    {lim}")
    return (l1, l2)
```

```
def summarize(sim):
    """Model outputs on the statistics window (warmup, T].

    p_skip           = N_skip / (N_fill + N_skip)
    qi_mean, qi_std  - time-weighted mean and std of the queue,
    items
    qi_max           - maximum queue held for a positive time,
    items
    qi_end           - queue at the end of the run, items
    sections_per_hour, items_per_hour - throughput (items of each
    type)
    lambda_i         - observed arrival rate of type i, items/min
    w_i              - mean waiting time of a type-i item by
    Little's law, min
    block_i          - share of time the type-i supplier waits for
    buffer room
                       (time-mean of the G_i marker; 0 without a
    buffer)
    """
    params = sim.net.params
    n_fill = sim.window_out[FILL]
    n_skip = sim.window_out[SKIP]
    total = n_fill + n_skip
    span = sim.t - sim.warmup
    q1 = sim.place_stats("Q1")
    q2 = sim.place_stats("Q2")
    lam1 = params["batch1"] * sim.window_out[ARRIVAL_1] / span
    lam2 = params["batch2"] * sim.window_out[ARRIVAL_2] / span
    nan = float("nan")
    return {
        "T": sim.t,
        "warmup": sim.warmup,
        "n_fill": n_fill,
        "n_skip": n_skip,
        "p_skip": n_skip / total if total else nan,
        "q1_mean": q1["mean"], "q1_std": q1["std"], "q1_max":
    q1["max"],
        "q2_mean": q2["mean"], "q2_std": q2["std"], "q2_max":
    q2["max"],
        "q1_end": sim.net.places["Q1"].markers,
        "q2_end": sim.net.places["Q2"].markers,
        "sections_per_hour": 60.0 * n_fill / span,
        "items_per_hour": 60.0 * params["k"] * n_fill / span,
        "lambda1": lam1,
        "lambda2": lam2,
        "w1": q1["mean"] / lam1 if lam1 > 0 else nan,
        "w2": q2["mean"] / lam2 if lam2 > 0 else nan,
        "block1": sim.place_stats("G1")["mean"],
        "block2": sim.place_stats("G2")["mean"],
    }
```

ДОДАТОК Б. Код верифікації моделі

Наведено протокол подій, обчислення аналітичних очікуваних значень, спільні функції серій прогонів і верифікаційні прогони.

Файл src/petri/trace.py

```
class EventProtocol:
    """Event log: time, phase (IN / OUT), transition, marking
    after the event."""

    def __init__(self, places=None, limit=None,
                 headers=("t", "phase", "transition"),
    phase_names=None):
        self.places = places  # place names to show; None = all
        self.limit = limit
        self.headers = headers
        self.phase_names = phase_names or {}
        self.rows = []

    @property
    def full(self):
        return self.limit is not None and len(self.rows) >=
    self.limit

    def record(self, t, phase, transition, net):
        if self.full:
            return
        marking = net.marking()
        if self.places is not None:
            marking = {p: marking[p] for p in self.places}
        self.rows.append((t, phase, transition.name, marking))

    def to_text(self):
        if not self.rows:
            return ""
        names = list(self.rows[0][3])
        h_t, h_phase, h_tr = self.headers
        phase = {ph: self.phase_names.get(ph, ph) for ph in ("IN",
    "OUT")}
        pw = max(len(h_phase), *(len(v) for v in phase.values()))
        width = max(len(h_tr), *(len(r[2]) for r in self.rows))
        head = f"{h_t:>8}  {h_phase:<{pw}}  {h_tr:<{width}}  " + "
    ".join(f"{n:>7}" for n in names)
        lines = [head]
        for t, ph, tr, marking in self.rows:
            lines.append(f"{t:8.3f}  {phase[ph]:<{pw}}
    {tr:<{width}}  "
                         + "  ".join(f"{marking[n]:>7}" for n in
    names))
        return "\n".join(lines)
```

Файл src/experiments/theory.py

```
"""Analytical estimates used to check the simulation.

Fluid (mean-rate) approximation. Arrival rates lambda_i = b_i /
    mu_i items/min,
conveyor capacity c = k / period items/min of each type. Filled
    sections
remove items of both types at the rate f = min(lambda_1, lambda_2,
    c), so
    P_skip  ~ 1 - f / c = 1 - lambda_min * period / k   (if
    lambda_min < c),
    slope_i = lambda_i - f   (linear queue growth, items/min),
and over [0, T] a linearly growing queue has mean ~ slope * T / 2
    and
maximum ~ slope * T.

Renewal theory. For batches b arriving after intervals with mean
    mu and
variance s^2 (first batch after one interval)
    E[A(t)] -> b * t / mu + b * (s^2 - mu^2) / (2 mu^2).
Since Q1(t) - Q2(t) = A1(t) - A2(t), the time-mean difference of
    the queues is
    Q1_mean - Q2_mean ~ (lambda_1 - lambda_2) * T / 2 + off_1 -
    off_2.
"""
import math

from src.petri.rng import Const, Unif

def interval_moments(interval):
    if isinstance(interval, Const):
        return interval.d, 0.0
    if isinstance(interval, Unif):
        return interval.d, (2 * interval.s) ** 2 / 12
    raise TypeError(f"no moments for {interval!r}")

def rate(batch, interval):
    mu, _ = interval_moments(interval)
    return batch / mu

def renewal_offset(batch, interval):
    mu, var = interval_moments(interval)
    return batch * (var - mu ** 2) / (2 * mu ** 2)

def fluid(k, period, lam1, lam2):
    c = k / period
    f = min(lam1, lam2, c)
    return {
        "capacity": c,
        "fill_rate": f,
        "p_skip": 1 - f / c,
```

```
        "slope1": lam1 - f,
        "slope2": lam2 - f,
        "rho": max(lam1, lam2) / c,
    }

def queue_difference(T, batch1, interval1, batch2, interval2):
    lam1, lam2 = rate(batch1, interval1), rate(batch2, interval2)
    return (lam1 - lam2) * T / 2 + renewal_offset(batch1,
    interval1) - renewal_offset(batch2, interval2)

def difference_variance_rate(batch1, interval1, batch2,
    interval2):
    """D = sum b^2 s^2 / mu^3: Var[A1(t) - A2(t)] ~ D * t,
    items^2/min."""
    d = 0.0
    for b, itv in ((batch1, interval1), (batch2, interval2)):
        mu, var = interval_moments(itv)
        d += b ** 2 * var / mu ** 3
    return d

def excess_growth_coef(batch1, interval1, batch2, interval2):
    """Time-mean over [0, T] of E|A1(t) - A2(t)| ~ coef * sqrt(T).

    With equal arrival rates A1 - A2 is a zero-drift random walk;
    for a normal
    value with variance D t, E|X| = sqrt(2 D t / pi), and the mean
    of sqrt(t)
    over [0, T] is 2/3 sqrt(T). Every filled section removes the
    same number of
    items of both types, so this excess stays in one of the queues
    whatever the
    conveyor speed is: if the conveyor keeps min(Q1, Q2) bounded,
    Q1_mean + Q2_mean ~ coef * sqrt(T) + const.
    """
    d = difference_variance_rate(batch1, interval1, batch2,
    interval2)
    return 2 / 3 * math.sqrt(2 * d / math.pi)
```

Файл src/experiments/common.py

```
import math
from concurrent.futures import ProcessPoolExecutor
from functools import partial

import numpy as np
from scipy import stats

from src.model.conveyor import build_conveyor_net, summarize
from src.petri import Simulator

def run_once(seed, T, warmup=0.0, protocol=None, **model_kwargs):
    net = build_conveyor_net(**model_kwargs)
```

```
    sim = Simulator(net, seed=seed, protocol=protocol).run(T,
    warmup=warmup)
    return summarize(sim)

def replicate(runs, T, seed=1, warmup=0.0, **model_kwargs):
    """Independent runs with seeds seed, seed+1, ...; list of
    summaries."""
    return [run_once(seed + i, T, warmup, **model_kwargs) for i in
    range(runs)]

def mean_ci(values, beta=0.95):
    """Sample mean and half-width of the Student confidence
    interval."""
    x = np.asarray(values, dtype=float)
    n = len(x)
    m = float(x.mean())
    if n < 2:
        return m, math.nan
    half = stats.t.ppf((1 + beta) / 2, n - 1) * x.std(ddof=1) /
    math.sqrt(n)
    return m, float(half)

def replicate_parallel(runs, T, seed=1, warmup=0.0,
    **model_kwargs):
    """Same as replicate(), runs spread over processes."""
    job = partial(run_once, T=T, warmup=warmup, **model_kwargs)
    with ProcessPoolExecutor() as ex:
        return list(ex.map(job, range(seed, seed + runs)))
```

Файл src/experiments/verification.py

```
"""Verification of the simulation algorithm.

python -m src.experiments.verification [--runs 100] [--T 2000]

Each row changes one or two inputs of the base model and compares
    the outputs
with an analytical expectation (src/experiments/theory.py) or, for
    fixed
arrival intervals, with a hand calculation. Writes
    results/verification.csv
    results/protocol_deterministic.txt           (full event
    protocol)
    results/protocol_deterministic_fragment.txt  (first events,
    for the paper)
"""
import argparse
import csv
import math
from pathlib import Path

from src.experiments.common import mean_ci, replicate_parallel,
    run_once
```

```
from src.experiments.theory import excess_growth_coef, fluid,
    queue_difference, rate
from src.petri import EventProtocol, const, unif

RESULTS = Path("results")

BASE = dict(k=10, period=10.0, batch1=5, interval1=unif(5, 1),
    batch2=20, interval2=unif(20, 7))
BASE_TEXT = ("k = 10 виробів/секцію, конвеєр 1 секція / 10 хв, тип
    1: 5 шт. кожні unif(5, 1) хв, "
             "тип 2: 20 шт. кожні unif(20, 7) хв, початкові черги
    0, збір статистики з 0 хв")

ROWS = [
    ("1", "базова модель", {}, "random"),
    ("2", "детерміновані інтервали надходження",
    dict(interval1=const(5), interval2=const(20)), "det"),
    ("3", "інтервал надходження типу 2 у 2 рази більший",
    dict(interval2=unif(40, 7)), "random"),
    ("4", "конвеєр у 2 рази швидший", dict(period=5.0), "random"),
    ("5", "конвеєр у 2 рази повільніший", dict(period=20.0),
    "random"),
    ("6", "варіант 20/20", dict(k=20, period=20.0), "random"),
    ("7", "партія типу 1 у 2 рази більша", dict(batch1=10),
    "random"),
]

PROTOCOL_PLACES = ["Q1", "Q2", "S", "N_fill", "N_skip"]
METRICS = ["p_skip", "q1_mean", "q2_mean", "q1_max", "q2_max"]

def describe(params):
    names = {"k": "k", "period": "період, хв", "batch1": "b1",
    "interval1": "інтервал 1, хв",
             "batch2": "b2", "interval2": "інтервал 2, хв"}
    return "; ".join(f"{names[key]} = {val!r}" if not
    isinstance(val, (int, float))
                     else f"{names[key]} = {val:g}" for key, val
    in params.items())

def deterministic_expectation(T):
    """Hand calculation for intervals exactly 5 and 20 min, k =
    10, period 10.

    t = 10: Q1 = 10, Q2 = 0  -> the only skipped section.
    From t = 20 every section is filled; Q1 is 10 on [20 + 10j, 25
    + 10j) and 15
    on [25 + 10j, 30 + 10j); Q2 is 10 on [20 + 20j, 30 + 20j) and
    0 on
    [30 + 20j, 40 + 20j). Before t = 20: Q1 = 0, 5, 10, 15 on 5-
    min steps, Q2 = 0.
    """
    sections = math.floor(T / 10)
```

```
    q1_area = (0 + 5 + 10 + 15) * 5 + 12.5 * (T - 20)
    q2_area = 5.0 * (T - 20)
    return {"p_skip": 1 / sections, "q1_mean": q1_area / T,
    "q2_mean": q2_area / T,
            "q1_max": 15, "q2_max": 10, "delta": (q1_area -
    q2_area) / T}

def analytic_expectation(params, T):
    lam1 = rate(params["batch1"], params["interval1"])
    lam2 = rate(params["batch2"], params["interval2"])
    fl = fluid(params["k"], params["period"], lam1, lam2)
    exp = {"lambda1": lam1, "lambda2": lam2, **fl,
           "delta": queue_difference(T, params["batch1"],
    params["interval1"],
                                     params["batch2"],
    params["interval2"]),
           "items_per_hour": 60 * fl["fill_rate"]}
    for i in (1, 2):
        s = fl[f"slope{i}"]
        exp[f"q{i}_mean"] = s * T / 2 if s > 0 else math.nan
        exp[f"q{i}_max"] = s * T if s > 0 else math.nan
    return exp

def save_protocol(T):
    proto = EventProtocol(places=PROTOCOL_PLACES, headers=("t,
    хв", "фаза", "перехід"),
                          phase_names={"IN": "вхід", "OUT":
    "вихід"})
    run_once(1, T, protocol=proto, **{**BASE, "interval1":
    const(5), "interval2": const(20)})
    text = proto.to_text()
    (RESULTS / "protocol_deterministic.txt").write_text(text +
    "\n", encoding="utf-8")
    fragment = "\n".join(text.splitlines()[:40])
    (RESULTS /
    "protocol_deterministic_fragment.txt").write_text(fragment +
    "\n", encoding="utf-8")
    return len(proto.rows)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--runs", type=int, default=100)
    ap.add_argument("--T", type=float, default=2000)
    a = ap.parse_args()
    RESULTS.mkdir(exist_ok=True)

    out = []
    for no, change, changed, kind in ROWS:
        params = {**BASE, **changed}
        runs = 1 if kind == "det" else a.runs
        res = replicate_parallel(runs, a.T, **params)
```

```
    row = {"no": no, "change": change, "inputs":
describe(changed), "T": a.T, "runs": runs}
    for key in METRICS + ["items_per_hour"]:
        row[key], row[key + "_ci"] = mean_ci([r[key] for r in
res])
    row["delta"], row["delta_ci"] = mean_ci([r["q1_mean"] -
r["q2_mean"] for r in res])
    for i in (1, 2):
        row[f"slope{i}"], row[f"slope{i}_ci"] =
mean_ci([r[f"q{i}_end"] / a.T for r in res])

    exp = deterministic_expectation(a.T) if kind == "det" else
analytic_expectation(params, a.T)
    for key, val in exp.items():
        row["exp_" + key] = val

    # underloaded conveyor: min(Q1, Q2) stays bounded, but the
excess
    # |A1 - A2| of one type grows like sqrt(T)
(theory.excess_growth_coef);
    # compare the growth of Q1_mean + Q2_mean between T and 5T
    if kind == "random" and exp["rho"] < 1:
        long = replicate_parallel(runs, 5 * a.T, **params)
        for key in ("q1_mean", "q2_mean"):
            row[key + "_5T"], row[key + "_5T_ci"] =
mean_ci([r[key] for r in long])
        growth = [(l["q1_mean"] + l["q2_mean"]) -
(s_["q1_mean"] + s_["q2_mean"])
                  for s_, l in zip(res, long)]
        row["sum_growth"], row["sum_growth_ci"] =
mean_ci(growth)
        coef = excess_growth_coef(params["batch1"],
params["interval1"],
                                  params["batch2"],
params["interval2"])
        row["exp_sum_growth"] = coef * (math.sqrt(5 * a.T) -
math.sqrt(a.T))
    out.append(row)

keys = []
for r in out:
    keys += [k for k in r if k not in keys]
with (RESULTS / "verification.csv").open("w", newline="",
encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=keys,
lineterminator="\n")
    w.writeheader()
    w.writerows(out)

n_events = save_protocol(a.T)

print(f"Незмінні значення: {BASE_TEXT}")
```

```
    print(f"T = {a.T:g} хв; прогонів: {a.runs} (детермінований
    рядок – 1)\n")
    for r in out:
        print(f"[{r['no']}] {r['change']}  ({r['inputs'] or 'без
    змін'})")
        for key in METRICS + ["delta", "slope1", "slope2",
    "items_per_hour"]:
            ci = r.get(key + "_ci", math.nan)
            e = r.get("exp_" + key, math.nan)
            ci_s = "" if math.isnan(ci) else f" ± {ci:.4f}"
            e_s = "" if e is None or (isinstance(e, float) and
    math.isnan(e)) else f"   очікується {e:.4f}"
            print(f"    {key:15} {r[key]:11.4f}{ci_s}{e_s}")
        for key in ("q1_mean_5T", "q2_mean_5T"):
            if key in r:
                print(f"    {key:15} {r[key]:11.4f} ± {r[key +
    '_ci']:.4f}   (T = {5 * a.T:g})")
        if "sum_growth" in r:
            print(f"    {'sum_growth':15} {r['sum_growth']:11.4f}
    ± {r['sum_growth_ci']:.4f}"
                  f"   очікується {r['exp_sum_growth']:.4f}
    (приріст Q1+Q2 від T до 5T)")
    print(f"\nПротокол детермінованого сценарію: {n_events} подій
    -> results/protocol_deterministic.txt")

if __name__ == "__main__":
    main()
```

ДОДАТОК В. Код визначення перехідного періоду та кількості прогонів

Функції побудови графіків і запису результатів у файли CSV, а також функції main, які лише розбирають параметри командного рядка і викликають наведені функції, не наведено; повний код розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation-course-work.

Файл src/experiments/transient.py

```
"""Transient period study (guide 3.1.4, criterion 3.1).

python -m src.experiments.transient

Base model and variant 20/20 (no buffer): running means of 5 runs
    over a long
horizon. The queues do not settle (load factor 1, Q1 - Q2 is a
    zero-drift
random walk), so criterion (3.1) is not reached for them.

Buffer model (kanban):
1) warm-up - ensemble mean over many runs of the time-weighted
    queue in short
   windows, for the grid corner with the largest buffers (slowest
    to settle);
2) run length - running means of 5 runs collected after the warm-
    up, for the
   point of the grid where the decision P_skip <= level is taken;
    criterion (3.1).
"""
import csv
import math
from concurrent.futures import ProcessPoolExecutor
from functools import partial
from itertools import combinations
from pathlib import Path

import numpy as np

from src.model.conveyor import FILL, SKIP, build_conveyor_net
from src.petri import Simulator
from src.plots.style import RUN_COLORS, decimal_comma, plt

RESULTS = Path("results")

N_RUNS = 5
BASE_T, BASE_STEP = 20000, 50

BUF_CORNER = (30, 80)      # largest buffers of the series grid
```

```
BUF_DECISION = (20, 60)    # region where P_skip crosses the
    chosen level
ENS_RUNS, ENS_T, ENS_STEP = 400, 6000, 50
RUN_T, RUN_STEP = 100000, 100
T_MOD_BUF = 40000          # chosen run length of the buffer model
    (warm-up 3000 min)
# eps_T of criterion (3.1): run-to-run spread of single-run means;
    the final
# accuracy of the estimates is reached by averaging p runs
    (replications.py)
EPS = {"p_skip": 0.005, "q1_mean": 2.0, "q2_mean": 2.0}

LABELS = {"q1_mean": "Середня довжина черги\nвиробів типу 1, од.",
          "q2_mean": "Середня довжина черги\nвиробів типу 2, од.",
          "p_skip": "Ймовірність пропуску\nсекції, –"}

def trace_run(seed, T, step, warmup=0.0, **model):
    """Running (cumulative) means and window means on a time grid.

    q1, q2 are time-weighted means over each grid window (t -
    step, t].
    Instantaneous markings are not used: the grid points are
    multiples of the
    conveyor period, where the queues sit at their pre-fill
    maximum.
    """
    rows = []

    def sample(sim):
        n_fill, n_skip = sim.window_out[FILL],
    sim.window_out[SKIP]
        q1, q2 = sim.net.places["Q1"].stat,
    sim.net.places["Q2"].stat
        rows.append((sim.t, q1.area, q2.area, q1.mean, q2.mean,
                     n_skip / (n_fill + n_skip) if n_fill + n_skip
    else math.nan))

    Simulator(build_conveyor_net(**model), seed=seed).run(
        T, warmup=warmup, sample_every=step, on_sample=sample)
    a = np.array(rows, dtype=float)
    win = np.diff(np.concatenate(([0.0], a[:, 0])))
    win[a[:, 0] <= warmup] = math.nan
    out = {"t": a[:, 0],
           "q1": np.diff(np.concatenate(([0.0], a[:, 1]))) / win,
           "q2": np.diff(np.concatenate(([0.0], a[:, 2]))) / win,
           "q1_mean": a[:, 3], "q2_mean": a[:, 4], "p_skip": a[:,
    5]}
    keep = out["t"] > warmup
    return {k: v[keep] for k, v in out.items()}

def traces(seeds, T, step, warmup=0.0, **model):
    with ProcessPoolExecutor() as ex:
```

```
        return list(ex.map(partial(trace_run, T=T, step=step,
    warmup=warmup, **model), seeds))

def max_pair_diff(runs, key):
    return np.max([np.abs(a[key] - b[key]) for a, b in
    combinations(runs, 2)], axis=0)

def settle_time(t, diff, eps):
    """First grid time after which the spread stays below eps
    (criterion 3.1)."""
    bad = np.where(~(diff < eps))[0]
    if len(bad) == 0:
        return t[0]
    return math.inf if bad[-1] == len(t) - 1 else t[bad[-1] + 1]

def base_study(name, model, file):
    runs = traces(range(1, N_RUNS + 1), BASE_T, BASE_STEP,
    **model)
    t = runs[0]["t"]
    rows = []
    for T in (480, 2400, 5000, 10000, 20000):
        i = np.searchsorted(t, T)
        rows.append({"model": name, "T": T,
                     **{f"spread_{k}": float(max_pair_diff(runs,
    k)[i]) for k in LABELS},
                     **{f"mean_{k}": float(np.mean([r[k][i] for r
    in runs])) for k in LABELS}})
    plot_running(runs, RESULTS / file)
    return rows

def warmup_study(buffer):
    ens = traces(range(1, ENS_RUNS + 1), ENS_T, ENS_STEP,
    buffer=buffer)
    t = ens[0]["t"]
    q1 = np.mean([r["q1"] for r in ens], axis=0)
    q2 = np.mean([r["q2"] for r in ens], axis=0)
    tail = t > ENS_T * 2 / 3
    lvl1, lvl2 = q1[tail].mean(), q2[tail].mean()
    # warm-up ends when the ensemble mean stays within 5 % of the
    steady level
    ok = (np.abs(q1 - lvl1) <= 0.05 * lvl1) & (np.abs(q2 - lvl2)
    <= 0.05 * lvl2)
    bad = np.where(~ok)[0]
    t_raw = float(t[bad[-1] + 1]) if len(bad) else float(t[0])
    t_warm = float(math.ceil(t_raw / 1000) * 1000)

    fig, ax = plt.subplots(figsize=(7.5, 3.6))
    ax.plot(t, q1, color=RUN_COLORS[0], lw=1, label="черга виробів
    типу 1")
    ax.plot(t, q2, color=RUN_COLORS[1], lw=1, label="черга виробів
    типу 2")
    for lvl, c in ((lvl1, RUN_COLORS[0]), (lvl2, RUN_COLORS[1])):
```

```
            ax.axhspan(0.95 * lvl, 1.05 * lvl, color=c, alpha=0.12)
        ax.axvline(t_warm, color="k", ls="--", lw=1, label=f"кінець
        перехідного періоду, {t_warm:g} хв")
        ax.set_xlabel(r"Модельний час $t$, хв")
        ax.set_ylabel(f"Середня за {ENS_RUNS} прогонами\nдовжина
        черги, од.")
        ax.set_ylim(bottom=0)
        ax.legend(fontsize=9, loc="lower right")
        decimal_comma(ax)
        fig.tight_layout()
        fig.savefig(RESULTS / "transient_buffer_warmup.png")
        plt.close(fig)
        return {"level_q1": lvl1, "level_q2": lvl2, "t_warm_raw":
        t_raw, "t_warm": t_warm}

def run_length_study(buffer, t_warm, file=None):
        runs = traces(range(1, N_RUNS + 1), RUN_T, RUN_STEP,
        warmup=t_warm, buffer=buffer)
        t = runs[0]["t"]
        settle = {k: float(settle_time(t, max_pair_diff(runs, k), e))
        for k, e in EPS.items()}
        final = {k: float(max_pair_diff(runs, k)[-1]) for k in EPS}
        if file:
            plot_running(runs, RESULTS / file, x0=t_warm,
        t_mod=T_MOD_BUF)
        return settle, final
```

Файл src/experiments/replications.py

```
"""Number of runs by formula (3.5): p = 20 * (sigma / eps)^2, beta
    = 0.95.

python -m src.experiments.replications

sigma is the standard deviation of a response over pilot runs.
Terminating study (base model and variant 20/20): T_mod = 480 min
    (one shift),
2400 min (one week) as a sensitivity check.
Buffer model: T_mod = 40000 min, statistics after a 3000-min warm-
    up, at the
decision point (20, 60) and at the grid corner with the largest
    buffers (30, 80).
"""
import csv
import math
from pathlib import Path

import numpy as np

from src.experiments.common import replicate_parallel

RESULTS = Path("results")
BETA = 0.95
```

```
EPS_TERM = {"p_skip": 0.005, "q1_mean": 1.0, "q2_mean": 1.0,
    "q1_max": 2.0, "q2_max": 2.0}
EPS_BUF = {"p_skip": 0.002, "q1_mean": 1.0, "q2_mean": 1.0}
T_BUF, WARM_BUF = 40000, 3000

STUDIES = [
    ("10/10", 480, 0.0, 100, dict(), EPS_TERM),
    ("20/20", 480, 0.0, 100, dict(k=20, period=20.0), EPS_TERM),
    ("10/10", 2400, 0.0, 100, dict(), EPS_TERM),
    ("20/20", 2400, 0.0, 100, dict(k=20, period=20.0), EPS_TERM),
    ("буфер 20/60", T_BUF, WARM_BUF, 20, dict(buffer=(20, 60)),
    EPS_BUF),
    ("буфер 30/80", T_BUF, WARM_BUF, 20, dict(buffer=(30, 80)),
    EPS_BUF),
]

def runs_needed(sigma, eps, beta=BETA):
    """Formula (3.4): p = sigma^2 / (eps^2 (1 - beta)); (3.5) for
    beta = 0.95."""
    return math.ceil(sigma ** 2 / (eps ** 2 * (1 - beta)))

def main():
    rows = []
    for name, T, warm, n0, model, eps in STUDIES:
        pilot = replicate_parallel(n0, T, seed=10001, warmup=warm,
    **model)
        for key, e in eps.items():
            vals = np.array([r[key] for r in pilot], dtype=float)
            sigma = float(vals.std(ddof=1))
            rows.append({"model": name, "T": T, "warmup": warm,
    "pilot_runs": n0, "response": key,
                         "mean": float(vals.mean()), "sigma":
    sigma, "eps": e,
                         "p": max(runs_needed(sigma, e), 20)})
    with (RESULTS / "replications.csv").open("w", newline="",
    encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]),
    lineterminator="\n")
        w.writeheader()
        w.writerows(rows)
    for r in rows:
        print(f"{r['model']:12} T={r['T']:6} {r['response']:8}
    середнє {r['mean']:9.4f}  "
              f"σ {r['sigma']:8.4f}  ε {r['eps']:6g}  p =
    {r['p']}")

if __name__ == "__main__":
    main()
```

ДОДАТОК Г. Код експериментального дослідження

Функції побудови графіків, запису результатів у файли CSV і виведення результатів на екран (main, practical) не наведено; повний код – у репозиторії https://github.com/kqjqxb/Computer-simulation-course-work.

Файл src/experiments/queue_offset.py

```
"""Difference of the mean queues Q1 - Q2 compared with renewal
    theory.

python -m src.experiments.queue_offset [--runs 200] [--T 10000]
"""
import argparse
import csv
from pathlib import Path

from src.experiments.common import mean_ci, replicate_parallel
from src.experiments.theory import queue_difference
from src.petri import unif

RESULTS = Path("results")

CASES = [
    ("базовий: 5 шт. unif(5, 1) / 20 шт. unif(20, 7)",
    dict(batch2=20, interval2=unif(20, 7))),
    ("обидва типи: 5 шт. unif(5, 1)", dict(batch2=5,
    interval2=unif(5, 1))),
    ("тип 2: 20 шт. unif(20, 1)", dict(batch2=20,
    interval2=unif(20, 1))),
]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--runs", type=int, default=200)
    ap.add_argument("--T", type=float, default=10000)
    a = ap.parse_args()

    rows = []
    for name, kw in CASES:
        res = replicate_parallel(a.runs, a.T, **kw)
        m, h = mean_ci([r["q1_mean"] - r["q2_mean"] for r in res])
        th = queue_difference(a.T, 5, unif(5, 1), kw["batch2"],
    kw["interval2"])
        rows.append({"case": name, "T": a.T, "runs": a.runs,
    "delta": m, "delta_ci": h, "theory": th})
        print(f"{name:50}  Q1-Q2 = {m:7.3f} ± {h:6.3f}   теорія
    {th:7.3f}")

    RESULTS.mkdir(exist_ok=True)
```

```
    with (RESULTS / "queue_offset.csv").open("w", newline="",
    encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]),
    lineterminator="\n")
        w.writeheader()
        w.writerows(rows)

if __name__ == "__main__":
    main()
```

Файл src/experiments/estimates.py

```
"""Statistical estimates of the base model outputs (scheme 10/10).

python -m src.experiments.estimates [--T 480 2400] [--runs 1600]

Terminating simulation from empty queues, horizon T. For every
    output the
sample over p independent runs gives the mean, the standard
    deviation, the
95 % Student confidence interval of the mean and the smallest and
    largest
value over the runs. Seeds 1..p are the ones of the 10/10 level in
    anova.py,
so the means coincide with results/anova_T*.csv.
"""
import argparse
import csv
import math
from pathlib import Path

import numpy as np
from scipy import stats

from src.experiments.anova import RESPONSES
from src.experiments.common import replicate_parallel

RESULTS = Path("results")
BETA = 0.95

def estimate(values, beta=BETA):
    x = np.asarray(values, dtype=float)
    n = len(x)
    mean = float(x.mean())
    std = float(x.std(ddof=1))
    half = float(stats.t.ppf((1 + beta) / 2, n - 1) * std /
    math.sqrt(n))
    return {"mean": mean, "std": std, "ci": half, "ci_low": mean -
    half,
            "ci_high": mean + half, "min": float(x.min()), "max":
    float(x.max())}
```

```
def run(T, runs, seed=1):
    data = replicate_parallel(runs, T, seed=seed, k=10,
    period=10.0)
    rows = []
    for key, label in RESPONSES:
        row = {"T": T, "runs": runs, "response": key, "label":
    label}
        row.update(estimate([d[key] for d in data]))
        rows.append(row)
    return rows
```

Файл src/experiments/anova.py

```
"""One-factor analysis of variance: base scheme 10/10 vs variant
    20/20.

python -m src.experiments.anova [--T 480] [--runs 1600]

Terminating simulation from empty queues, horizon T (one shift by
    default).
The factor is the conveyor scheme (q = 2 levels), each level has p
    runs.

    D_f = p * sum_j (y_j - y)^2 / (q - 1)            factor
    variance
    D_r = sum_j sum_s (y_js - y_j)^2 / (q (p - 1))   residual
    variance
    F = D_f / D_r,  compared with F_crit(alpha = 0.05; q - 1, q (p
    - 1)).

The guide's formula (3.10) divides the factor sum of squares by q
    * p - 1;
the standard one-factor ANOVA (used here, and by
    scipy.stats.f_oneway, which
is run as a cross-check) divides it by q - 1. Formula (3.11) is
    the standard one.

ANOVA needs independent samples, so the levels use disjoint seed
    ranges
(1..p and 1000001..). With the same seeds the first arrival
    intervals of both
variants come from the same random numbers and the paired outputs
    correlate
(about +0.3), which breaks that assumption.
"""
import argparse
import csv
import math
from pathlib import Path

import numpy as np
from scipy import stats
```

```
from src.experiments.common import replicate_parallel

RESULTS = Path("results")
ALPHA = 0.05

VARIANTS = [("10/10", dict(k=10, period=10.0)), ("20/20",
    dict(k=20, period=20.0))]
RESPONSES = [
    ("p_skip", "Ймовірність пропуску секції, –"),
    ("q1_mean", "Середня довжина черги виробів типу 1, од."),
    ("q2_mean", "Середня довжина черги виробів типу 2, од."),
    ("q1_max", "Максимальна довжина черги виробів типу 1, од."),
    ("q2_max", "Максимальна довжина черги виробів типу 2, од."),
    ("items_per_hour", "Пропускна здатність, виробів/год"),
]

def oneway_anova(groups, alpha=ALPHA):
    """groups: list of equal-size samples. Own formulas + scipy
    cross-check."""
    y = [np.asarray(g, dtype=float) for g in groups]
    q, p = len(y), len(y[0])
    means = np.array([g.mean() for g in y])
    grand = means.mean()
    ss_f = p * np.sum((means - grand) ** 2)
    ss_r = sum(np.sum((g - m) ** 2) for g, m in zip(y, means))
    df_f, df_r = q - 1, q * (p - 1)
    d_f, d_r = ss_f / df_f, ss_r / df_r
    f = d_f / d_r if d_r > 0 else math.inf
    f_crit = stats.f.ppf(1 - alpha, df_f, df_r)
    sp = stats.f_oneway(*y)
    return {"D_f": d_f, "D_r": d_r, "df_f": df_f, "df_r": df_r,
    "F": f, "F_crit": f_crit,
            "p_value": float(stats.f.sf(f, df_f, df_r)),
    "significant": bool(f > f_crit),
            "F_scipy": float(sp.statistic), "p_scipy":
    float(sp.pvalue)}

def diff_ci(a, b, beta=0.95):
    """Mean of b - a with a Welch confidence interval."""
    a, b = np.asarray(a, float), np.asarray(b, float)
    va, vb = a.var(ddof=1) / len(a), b.var(ddof=1) / len(b)
    se = math.sqrt(va + vb)
    df = (va + vb) ** 2 / (va ** 2 / (len(a) - 1) + vb ** 2 /
    (len(b) - 1)) if se > 0 else 1
    return float(b.mean() - a.mean()), float(stats.t.ppf((1 +
    beta) / 2, df) * se)

SEED_STEP = 1_000_000

def run(T, runs, seed=1):
    data = {name: replicate_parallel(runs, T, seed=seed + i *
    SEED_STEP, **kw)
```

```
            for i, (name, kw) in enumerate(VARIANTS)}
    (na, ra), (nb, rb) = [(n, data[n]) for n, _ in VARIANTS]
    rows = []
    for key, label in RESPONSES:
        a = [r[key] for r in ra]
        b = [r[key] for r in rb]
        res = oneway_anova([a, b])
        d, h = diff_ci(a, b)
        rows.append({"T": T, "runs": runs, "response": key,
    "label": label,
                     "mean_" + na: float(np.mean(a)), "mean_" +
    nb: float(np.mean(b)),
                     "diff": d, "diff_ci": h,
                     "pair_corr": float(np.corrcoef(a, b)[0, 1]),
    **res})
    return rows
```

Файл src/experiments/buffer_series.py

```
"""Buffer (kanban) model in steady state: series over the buffer
    sizes (L1, L2).

python -m src.experiments.buffer_series [--runs 120] [--level
    0.02]

T_mod = 40000 min, statistics after a 3000-min warm-up
    (transient.py),
p = 120 runs per grid point (replications.py), seeds 1..p at every
    point.

1) Full search over the grid: the minimal total buffer L1 + L2 for
    which the
   upper bound of the 95 % confidence interval of P_skip does not
    exceed the level.
2) Regression of P_skip on L2 (L1 fixed): factor normalized to [-
    1; 1] by (3.6),
   coefficients b = (X^T X)^-1 X^T Y (3.9), correlation index R.
    It is fitted
   on L2 >= 40: L2 = 30 is a separate regime (30 = one type-2
    batch + one
   section, the next batch cannot start until a section takes
    items, P jumps
   from 0.04 to 0.20), outside the region of interest P <= 0.05.

Cost of the buffer: block_i - share of time the type-i supplier
    waits for room;
lost production = 60 * lambda_i * block_i items/h of each type.
"""
import argparse
import csv
import math
from pathlib import Path
```

```
import numpy as np

from src.experiments.common import mean_ci, replicate_parallel
from src.plots.style import RUN_COLORS, decimal_comma, plt

RESULTS = Path("results")
T_MOD, WARMUP = 40000, 3000
L1_GRID = (15, 20, 25, 30)
L2_GRID = (30, 40, 50, 60, 70, 80)
LAMBDA = 1.0  # nominal supply of each type, items/min
KEYS = ["p_skip", "q1_mean", "q2_mean", "q1_max", "q2_max",
    "items_per_hour", "block1", "block2"]

def grid_study(runs):
    rows = []
    for l1 in L1_GRID:
        for l2 in L2_GRID:
            res = replicate_parallel(runs, T_MOD, warmup=WARMUP,
    buffer=(l1, l2))
            row = {"L1": l1, "L2": l2, "runs": runs, "T": T_MOD,
    "warmup": WARMUP}
            for k in KEYS:
                row[k], row[k + "_ci"] = mean_ci([r[k] for r in
    res])
            row["q1_max_all"] = max(r["q1_max"] for r in res)
            row["q2_max_all"] = max(r["q2_max"] for r in res)
            for i in (1, 2):
                row[f"lost{i}_per_hour"] = 60 * LAMBDA *
    row[f"block{i}"]
            rows.append(row)
            print(f"L = ({l1}, {l2}): P = {row['p_skip']:.4f} ±
    {row['p_skip_ci']:.4f}, "
                  f"Q1 = {row['q1_mean']:.2f}, Q2 =
    {row['q2_mean']:.2f}, "
                  f"блок.
    {row['block1']:.4f}/{row['block2']:.4f}")
    return rows

def best_buffer(rows, level):
    ok = [r for r in rows if r["p_skip"] + r["p_skip_ci"] <=
    level]
    return min(ok, key=lambda r: (r["L1"] + r["L2"], r["p_skip"]))
    if ok else None

def normalize(x):
    """(3.6): x_norm = (X - X0) / delta,  X0 = (Xmax + Xmin) / 2,
    delta = (Xmax - Xmin) / 2."""
    x = np.asarray(x, dtype=float)
    x0, delta = (x.max() + x.min()) / 2, (x.max() - x.min()) / 2
    return (x - x0) / delta, x0, delta

def least_squares(columns, y):
```

```
    """(3.9): b = (X^T X)^-1 X^T Y."""
    X = np.column_stack([np.ones(len(y))] + list(columns))
    return np.linalg.solve(X.T @ X, X.T @ y), X

def correlation_index(y, y_hat):
    return math.sqrt(max(0.0, 1 - np.sum((y - y_hat) ** 2) /
    np.sum((y - y.mean()) ** 2)))

REG_L2_MIN = 40

def regression(rows, l1, l2_min=REG_L2_MIN):
    pts = sorted((r for r in rows if r["L1"] == l1 and r["L2"] >=
    l2_min), key=lambda r: r["L2"])
    L2 = np.array([r["L2"] for r in pts], dtype=float)
    P = np.array([r["p_skip"] for r in pts])
    x, x0, delta = normalize(L2)
    models = {
        "P = b0 + b1 x": (lambda: [x], False),
        "P = b0 + b1 x + b2 x^2": (lambda: [x, x ** 2], False),
        "ln P = b0 + b1 x": (lambda: [x], True),
        "ln P = b0 + b1 x + b2 x^2": (lambda: [x, x ** 2], True),
    }
    out = []
    for name, (cols, log) in models.items():
        target = np.log(P) if log else P
        b, X = least_squares(cols(), target)
        fit = X @ b
        p_hat = np.exp(fit) if log else fit
        out.append({"L1": l1, "L2_from": float(L2.min()), "L2_to":
    float(L2.max()),
                    "model": name, "x0": x0, "delta": delta,
                    **{f"b{i}": float(v) for i, v in
    enumerate(b)},
                    "R": correlation_index(P, p_hat),
                    "max_abs_err": float(np.max(np.abs(P -
    p_hat)))})
    return out, (L2, P, x0, delta)

def predict(model_row, L2):
    x = (np.asarray(L2, float) - model_row["x0"]) /
    model_row["delta"]
    b = [model_row.get(f"b{i}", 0.0) for i in range(3)]
    b = [0.0 if (isinstance(v, float) and math.isnan(v)) else v
    for v in b]
    y = b[0] + b[1] * x + b[2] * x ** 2
    return np.exp(y) if model_row["model"].startswith("ln") else y

def solve_level(model_row, level):
    """Continuous L2 with predicted P_skip = level (bisection on
    the fitted range)."""
    lo, hi = model_row["L2_from"], model_row["L2_to"]
    f = lambda v: float(predict(model_row, [v])[0]) - level
```

```
    if f(lo) * f(hi) > 0:
        return math.nan
    for _ in range(60):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if f(lo) * f(mid) > 0 else (lo, mid)
    return (lo + hi) / 2
```

Файл src/experiments/compare.py

```
"""Proposal (kanban buffer) compared with the base scheme and
    variant 20/20.

python -m src.experiments.compare [--T 480 2400] [--runs 1600] [--
    buffers 20 60 25 80]

1) Same conditions: empty queues at t = 0, horizon T, the same
    seeds 1..p for
   every model (common random numbers); differences to the base
    model are
   paired (run s of a model minus run s of the base), CI by
    Student.
2) Steady state: base model and the buffer models, T = 40000 min,
    warm-up 3000,
   120 runs - the buffers keep the queues bounded, the base model
    does not.

Two proposal variants: (20, 60) - minimal buffer with P_skip <=
    0.02,
(25, 80) - smaller loss of completeness (P_skip about 0.01).

Cost of the buffer: block_i (share of time the supplier waits) and
    the lost
production 60 * lambda_i * block_i items/h of each type (lambda_i
    = 1 item/min).
"""
import argparse
import csv
import math
from pathlib import Path

import numpy as np
from scipy import stats

from src.experiments.common import mean_ci, replicate_parallel
from src.plots.style import RUN_COLORS, decimal_comma, plt

RESULTS = Path("results")
KEYS = ["p_skip", "q1_mean", "q2_mean", "q1_max", "q2_max",
    "items_per_hour",
        "block1", "block2", "lost1", "lost2"]
STEADY_T, STEADY_WARM, STEADY_RUNS = 40000, 3000, 120
```

```
def models(buffers):
    return ([("база 10/10", dict()), ("варіант 20/20", dict(k=20,
    period=20.0))]
            + [(f"буфер ({b[0]}, {b[1]})", dict(buffer=tuple(b)))
    for b in buffers])

def with_lost(res):
    for r in res:
        r["lost1"] = 60.0 * r["block1"]
        r["lost2"] = 60.0 * r["block2"]
    return res

def paired_ci(a, b, beta=0.95):
    d = np.asarray(b, float) - np.asarray(a, float)
    h = stats.t.ppf((1 + beta) / 2, len(d) - 1) * d.std(ddof=1) /
    math.sqrt(len(d))
    return float(d.mean()), float(h)

def compare(T, runs, buffers, warmup=0.0, model_list=None):
    model_list = model_list or models(buffers)
    data = {name: with_lost(replicate_parallel(runs, T, seed=1,
    warmup=warmup, **kw))
            for name, kw in model_list}
    base = model_list[0][0]
    rows = []
    for name, kw in model_list:
        row = {"T": T, "warmup": warmup, "runs": runs, "model":
    name}
        for k in KEYS:
            vals = [r[k] for r in data[name]]
            row[k], row[k + "_ci"] = mean_ci(vals)
            if name != base:
                row["d_" + k], row["d_" + k + "_ci"] =
    paired_ci([r[k] for r in data[base]], vals)
        row["q1_max_all"] = max(r["q1_max"] for r in data[name])
        row["q2_max_all"] = max(r["q2_max"] for r in data[name])
        # per shift of 480 min: empty sections and items of each
    type not assembled
        # (a section carries k items, 480 / period sections a
    shift, k / period = 1 item/min)
        row["empty_sections_per_shift"] = row["p_skip"] * 480 /
    kw.get("period", 10.0)
        row["items_not_assembled_per_shift"] = row["p_skip"] * 480
        rows.append(row)
    return rows
```

ДОДАТОК Д. Фрагмент протоколу подій

Фрагмент протоколу подій детермінованого сценарію верифікації (рядок 2 таблиці 3.3): інтервали надходження 5 і 20 хв, k = 10, секція кожні 10 хв. У таблиці Д.1 наведено 36 подій на інтервалі (40; 80] хв, що продовжують таблицю 3.4; повний протокол містить 1803 подій. Маркування наведено після кожної фази.

Таблиця Д.1 – Протокол подій детермінованого сценарію

t, хв	Фаза	Перехід	Q_1	Q_2	S	N_{fill}	N_{skip}
45	вихід	Надходження 1	15	10	0	3	1
45	вхід	Надходження 1	15	10	0	3	1
50	вихід	Рух конвеєра	15	10	1	3	1
50	вихід	Надходження 1	20	10	1	3	1
50	вхід	Комплектація	10	0	0	3	1
50	вхід	Рух конвеєра	10	0	0	3	1
50	вхід	Надходження 1	10	0	0	3	1
50	вихід	Комплектація	10	0	0	4	1
55	вихід	Надходження 1	15	0	0	4	1
55	вхід	Надходження 1	15	0	0	4	1
60	вихід	Надходження 1	20	0	0	4	1
60	вихід	Надходження 2	20	20	0	4	1
60	вихід	Рух конвеєра	20	20	1	4	1
60	вхід	Комплектація	10	10	0	4	1
60	вхід	Надходження 2	10	10	0	4	1
60	вхід	Надходження 1	10	10	0	4	1
60	вхід	Рух конвеєра	10	10	0	4	1
60	вихід	Комплектація	10	10	0	5	1
65	вихід	Надходження 1	15	10	0	5	1
65	вхід	Надходження 1	15	10	0	5	1
70	вихід	Рух конвеєра	15	10	1	5	1
70	вихід	Надходження 1	20	10	1	5	1
70	вхід	Комплектація	10	0	0	5	1
70	вхід	Надходження 1	10	0	0	5	1
70	вхід	Рух конвеєра	10	0	0	5	1
70	вихід	Комплектація	10	0	0	6	1
75	вихід	Надходження 1	15	0	0	6	1
75	вхід	Надходження 1	15	0	0	6	1
80	вихід	Рух конвеєра	15	0	1	6	1
80	вихід	Надходження 1	20	0	1	6	1
80	вихід	Надходження 2	20	20	1	6	1
80	вхід	Комплектація	10	10	0	6	1
80	вхід	Надходження 1	10	10	0	6	1
80	вхід	Рух конвеєра	10	10	0	6	1
80	вхід	Надходження 2	10	10	0	6	1
80	вихід	Комплектація	10	10	0	7	1

ДОДАТОК Е. Результати перебору місткостей накопичувача

Результати отримано для моделі з накопичувачем, $T_{мод}$ = 40 000 хв, $T_п$ = 3000 хв, 120 прогонів у кожній точці. Середні значення ймовірності пропуску секції наведено в таблиці Е.1, середні довжини черг – на рисунку Е.1.

Таблиця Е.1 – Ймовірність пропуску секції в моделі з накопичувачем

L_1, од.	L_2 = 30 од.	L_2 = 40 од.	L_2 = 50 од.	L_2 = 60 од.	L_2 = 70 од.	L_2 = 80 од.
15	0,2009	0,0512	0,0322	0,0257	0,0230	0,0214
20	0,2003	0,0439	0,0229	0,0167	0,0134	0,0114
25	0,2003	0,0432	0,0213	0,0147	0,0116	0,0095
30	0,2003	0,0430	0,0212	0,0141	0,0110	0,0089

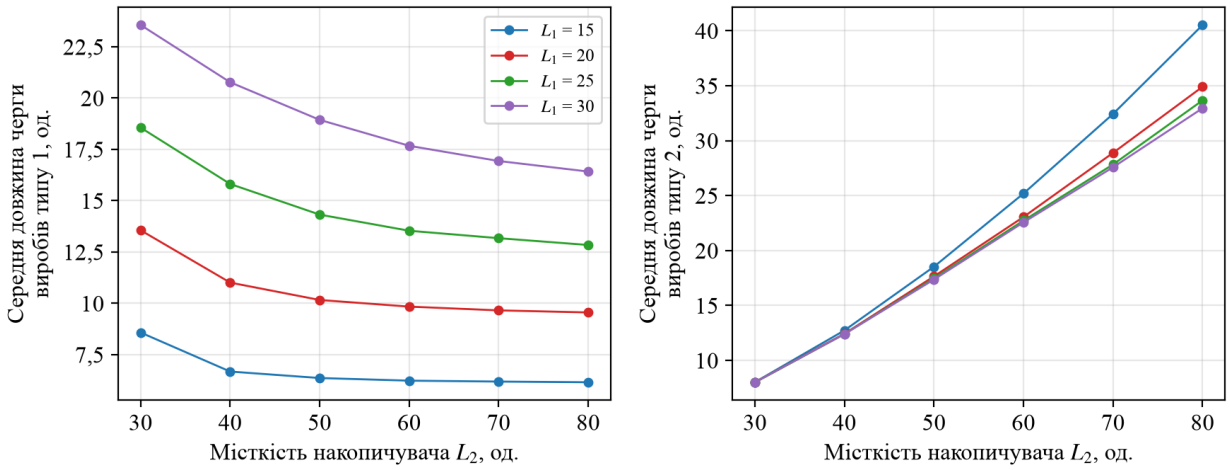


Рисунок Е.1 – Середні довжини черг залежно від місткостей